



Copy of MLP Level 1_2 ...
XLSX 180.59KB

this file contain questions of my viva , arrange the questions topic wise properly

MLP Viva Questions – Topic-wise Arrangement

Below is a comprehensive, topic-wise arrangement of all the questions asked across Level 1, Level 2, and Level 3 MLP vivas, compiled from the entire response sheet.

1. PROJECT & NOTEBOOK EXPLANATION

General Notebook Walkthrough

- Explain your notebook / project / problem statement.
- Explain the entire notebook workflow according to the rubrics.
- Explain what you have done in the notebook.
- Walk me through your notebook (EDA, preprocessing, feature engineering, models, HPT, final analysis).
- Go through the notebook and explain each and every step.
- Explain the problem statement and objectives (type of ML problem – classification/regression, supervised/unsupervised).
- What is the objective of this project?
- What was your approach to the problem?
- Explain the dataset and its features.
- Who created the dataset? What is the size of the dataset?
- What are the categorical and numerical variables?
- Show me your Kaggle submissions / leaderboard score / rank.
- Which notebook would you like to explain?
- Explain your project as if you are explaining to a non-technical person / your boss.
- What did you learn from this project?
- What difficulties/challenges did you face?

- How did you overcome those difficulties?
- What resources did you use?
- Did you use any LLM/AI? Where and why?
- What would you do differently if you had more time?
- How can you further improve your model/score?
- Future scope of the project.
- Real-world application of your project.

Specific Notebook Sections

- Show me your EDA / Feature Engineering / Preprocessing / Pipeline / Models / HPT / Model Comparison.
- Show me your graphs / visualizations.
- Explain the insights you gained from EDA.
- What is your best model and why?
- Show me your confusion matrix / classification report / ROC curve.

2. EXPLORATORY DATA ANALYSIS (EDA)

General EDA

- What is EDA and why do we do it?
- What all EDA have you used?
- Explain the graphs/visualizations you have plotted.
- What insights did you get from EDA?
- Pick any one graph and explain it in detail.
- What did you infer from the correlation matrix / heatmap?
- What is the use of a correlation matrix?
- How do you read a correlation matrix?
- How do you find correlation between categorical data?
- What is the range of correlation coefficient?
- What do correlation values -1, 0, and +1 represent?

- What is multicollinearity?
- If two columns are highly correlated, what should you do?
- How can you display correlation values directly on a heatmap?
- What is the difference between causality and correlation?
- What is the significance of the correlation between a feature and the target?
- If two features have 0 correlation, will they be collinear?
- How do you identify which feature affects the target most from the heatmap?
- What is the use of pair plots?
- How can you find non-linear relationships between features?

Specific Plots

- What is a box plot? What are its components (whiskers, quartiles, median, outliers)?
- What is the full name of a box plot? (Box-and-whisker plot)
- What do the points outside the box plot represent?
- What is a histogram? What does it show?
- Difference between a bar plot and a histogram.
- What is a scatter plot? What does it show?
- What is a density regression plot?
- What is a KDE plot? Why is it used?
- What is a violin plot?
- What is a pair plot?
- What is a word cloud? Where is it used?
- What is a line graph? When do you use it?
- What is a heatmap?
- What is a count plot?

Univariate & Bivariate Analysis

- What is univariate analysis?
- What is bivariate analysis?
- What is multivariate analysis?
- Show univariate and bivariate analysis from your notebook.

- What is the distribution of the target variable?
- Is the target variable normally distributed? How do you know?
- What is skewness? How do you handle skewed data?
- What is right-skewed and left-skewed data?
- What is log transformation? Why do we use it?
- What is `np.log1p` ? Difference between `log` and `log1p` .
- What is the purpose of `describe()` ?
- What insights do you get from `describe()` ?
- What is the difference between `unique()` and `nunique()` ?
- What is cardinality? How do you find high cardinality columns?

Outliers

- What are outliers?
- How do you detect outliers? (IQR, box plot, Z-score, etc.)
- What is IQR? How is it calculated?
- How do you handle outliers?
- Is removing outliers always good practice?
- What happens if you don't remove outliers?
- Which models are sensitive to outliers?
- Which models are robust to outliers?
- If there are outliers, which imputer should be used?
- What is the effect of outliers on mean, median, and mode?
- What is the limitation of mean?

Missing Values

- How do you handle missing values?
- What are the different techniques for handling missing values?
- What is imputation?
- Why do we do imputation?
- What is SimpleImputer? What are its strategies?
- When do you use mean, median, and mode for imputation?

- Why did you choose median over mean for imputation?
- What happens if you don't handle missing values?
- How do you handle missing values in categorical columns?
- What is the difference between `SimpleImputer` and `fillna` ?
- What is backward fill and forward fill?
- How do you handle disguised missing values like '?', 'None', 'Unknown'?
- Should you drop columns with many missing values or keep them?
- What percentage of missing values is acceptable before dropping a column?

Class Imbalance

- What is class imbalance?
- How do you detect class imbalance?
- How do you handle class imbalance?
- What is oversampling and undersampling?
- What is SMOTE?
- What is `class_weight='balanced'` ?
- Is accuracy a good metric for imbalanced data?
- What is the effect of class imbalance on model performance?
- How do you handle class imbalance in train-test split? (`stratify=y`)

Data Leakage

- What is data leakage?
- How do you prevent data leakage?
- Should `train_test_split` be done before or after preprocessing?
- Why should preprocessing be fitted only on training data?
- What is the difference between `fit_transform()` and `transform()` ?
- Why do we use `fit_transform()` on training data and only `transform()` on test data?

3. DATA PREPROCESSING

General Preprocessing

- What is data preprocessing?
- Why is preprocessing required?
- What are the steps in data preprocessing?
- What preprocessing did you do in your notebook?
- What is the order of preprocessing steps?
- What is a pipeline? Why do we use it?
- What is ColumnTransformer?
- Difference between Pipeline and ColumnTransformer.
- Difference between `fit()` and `fit_transform()` .
- What is the difference between `transform()` and `fit_transform()` ?
- What is `set_output(transform='pandas')` ?
- What is data cleaning?
- How do you handle duplicates?
- Why remove duplicates?
- How do you decide which duplicate to drop?

Feature Scaling

- What is feature scaling?
- Why is scaling important?
- What are the different scaling techniques?
- What is StandardScaler? How does it work?
- What is MinMaxScaler? How does it work?
- What is RobustScaler? How does it work?
- What is Normalization?
- Difference between Standardization and Normalization.
- Difference between StandardScaler and MinMaxScaler.
- Difference between StandardScaler and RobustScaler.
- What is the range of StandardScaler output?
- What is the range of MinMaxScaler output?
- What happens if you don't scale the data?

- Which models require scaling?
- Which models do not require scaling?
- Do tree-based models need scaling? Why or why not?
- What is the effect of outliers on StandardScaler vs MinMaxScaler?
- Which is more robust to outliers: StandardScaler or MinMaxScaler?
- What is the mean and standard deviation after StandardScaler?
- What percentage of values lie between -3σ and $+3\sigma$?
- What is 0-centering?
- What is the effect of scaling on model performance?
- Why is KNN sensitive to scaling?
- Why is scaling useful for Ridge Regression?

Feature Encoding

- What is encoding?
- Why do we need encoding?
- What are the different types of encoding?
- What is One-Hot Encoding (OHE)?
- What is Label Encoding?
- What is Ordinal Encoding?
- What is Target Encoding?
- What is Frequency Encoding?
- What is Binary Encoding?
- What is the difference between One-Hot Encoding and Label Encoding?
- What is the difference between One-Hot Encoding and Ordinal Encoding?
- What is the difference between Label Encoding and Target Encoding?
- When do you use Label Encoding?
- When do you use One-Hot Encoding?
- When do you use Ordinal Encoding?
- What is the issue with One-Hot Encoding?
- What is the issue with Label Encoding?
- What is high cardinality? How does it affect One-Hot Encoding?

- What is `handle_unknown='ignore'` in `OneHotEncoder`?
- What is `sparse_output=False` in `OneHotEncoder`?
- What is `drop='first'` in `OneHotEncoder`?
- What is the issue with Label Encoding for linear models?
- How do you handle high cardinality categorical features?
- Which encoding is best for high cardinality?
- What is `TargetEncoder` and how does it work?
- How do you prevent data leakage in target encoding?
- What is naive target encoding?
- What is smoothing in target encoding?
- What is OOF (Out-of-Fold) encoding?

Feature Engineering

- What is feature engineering?
- Why do we do feature engineering?
- What feature engineering did you do in your notebook?
- How did you decide which features to create?
- Did your feature engineering improve the score?
- How do you know if a new feature is useful?
- What is feature extraction?
- What is feature selection?
- Difference between feature engineering and feature selection.
- What are the different feature selection techniques?
- What is RFE (Recursive Feature Elimination)?
- How does RFE work?
- What is `SelectKBest`?
- What is chi-square test? How is it used for feature selection?
- What is ANOVA test?
- What is mutual information?
- What is the curse of dimensionality?
- What is dimensionality reduction?

- What is PCA? How does it work?
- Is PCA supervised or unsupervised?
- Is PCA linear or non-linear?
- What is kernel PCA?
- How many principal components should you choose?
- What is explained variance ratio?
- What is Truncated SVD?
- What is the difference between PCA and Truncated SVD?
- Does PCA discard data points?
- Can you get back the original data after PCA?
- What are eigenvalues and eigenvectors?
- How do you calculate PCA manually?
- How do you calculate eigenvectors manually?
- What is the relation between PCA and SVD?
- What is LDA?
- What is the difference between PCA and LDA?
- What is t-SNE?
- What is UMAP?

4. MACHINE LEARNING MODELS

General Model Questions

- Which models did you use?
- Why did you choose these models?
- How did you select the models?
- How many models did you train?
- How many models did you hyperparameter tune?
- Which model gave the best result and why?
- Why did this model perform better than others?
- What are the parameters of the models you used?

- What are the hyperparameters of the models you used?
- What is the difference between parameters and hyperparameters?
- What is the difference between parametric and non-parametric models?
- Which of your models are parametric?
- Which of your models are non-parametric?
- Give examples of parametric models.
- Give examples of non-parametric models.
- What is a baseline model?
- What is a weak learner?
- What is an ensemble model?
- What is bagging?
- What is boosting?
- Difference between bagging and boosting.
- Which reduces bias and which reduces variance?
- Does bagging reduce variance or both bias and variance?
- What is stacking?
- What is blending?
- What is voting classifier?
- What is the difference between bagging and bootstrapping?
- What is OOB (Out-of-Bag) score?
- What is the OOB score parameter in Random Forest?
- What is the difference between hard voting and soft voting?

Linear Models

- What is Linear Regression?
- How does Linear Regression work?
- What is the loss function of Linear Regression?
- What are the assumptions of Linear Regression?
- What is the limitation of Linear Regression?
- What is Logistic Regression?
- How does Logistic Regression work?

- Why is Logistic Regression called "regression" when it is used for classification?
- What is the loss function of Logistic Regression?
- What is log loss?
- What is the sigmoid function?
- What is the formula of the sigmoid function?
- What is the range of the sigmoid function?
- What is the threshold in Logistic Regression?
- How do you choose the threshold?
- How does Logistic Regression handle multi-class problems?
- What is One-vs-Rest (OvR)?
- What is One-vs-One (OvO)?
- What is the difference between OvR and OvO?
- What is the optimization algorithm in Logistic Regression?
- What is Gradient Descent?
- What is Stochastic Gradient Descent (SGD)?
- Difference between Batch GD and Stochastic GD.
- What is the learning rate?
- What happens if the learning rate is too high or too low?
- What is the effect of learning rate on overfitting/underfitting?
- What is regularization?
- What is L1 regularization (Lasso)?
- What is L2 regularization (Ridge)?
- Difference between L1 and L2 regularization.
- Which one is used for feature selection?
- Why does L1 help in eliminating features?
- Why does L2 prevent overfitting?
- What is ElasticNet?
- What is the penalty term in Ridge and Lasso?
- What is the alpha parameter in Ridge/Lasso?
- What is the C parameter in Logistic Regression?
- What is the `solver` parameter in Logistic Regression?

- What is `max_iter` in Logistic Regression?
- What is `class_weight` in Logistic Regression?

Tree-Based Models

- What is a Decision Tree?
- How does a Decision Tree work?
- How is the root node selected in a Decision Tree?
- How are splits decided in a Decision Tree?
- What is entropy?
- What is information gain?
- What is the Gini index?
- What is the range of the Gini index?
- What is the difference between entropy and Gini?
- What is a pure node and an impure node?
- What is the depth of a Decision Tree?
- What is the height of a Decision Tree?
- How do you prevent overfitting in a Decision Tree?
- What is `max_depth` ?
- What is `min_samples_split` ?
- What is `min_samples_leaf` ?
- What is pruning?
- What is pre-pruning and post-pruning?
- What is Random Forest?
- How does Random Forest work?
- Is Random Forest a bagging or boosting algorithm?
- Why is it called "Random" Forest?
- What is feature sampling in Random Forest?
- What is bootstrapping in Random Forest?
- What are the parameters of Random Forest?
- What is `n_estimators` ?
- What happens if you increase `n_estimators` ?

- What happens if you decrease `n_estimators` ?
- What is the effect of `max_depth` in Random Forest?
- What is the effect of `max_features` ?
- What is the OOB score?
- Why does Random Forest tend to overfit?
- How do you prevent overfitting in Random Forest?
- What is the difference between Random Forest and Decision Tree?
- What is the difference between Random Forest and Extra Trees?
- What is the difference between Random Forest and Gradient Boosting?
- What is the difference between Random Forest and XGBoost?
- What is the loss function in Random Forest?
- What is the regularization method for Random Forest?
- How does Random Forest handle missing values?
- How does Random Forest handle categorical features?

Boosting Models

- What is Gradient Boosting?
- How does Gradient Boosting work?
- What is XGBoost?
- How does XGBoost work?
- What is the difference between XGBoost and Gradient Boosting?
- What is the difference between XGBoost and LightGBM?
- What is the difference between XGBoost and CatBoost?
- What is the difference between LightGBM and CatBoost?
- What is the difference between XGBoost and Random Forest?
- What is the difference between LightGBM and XGBoost?
- Why is LightGBM called "Light"?
- Why is XGBoost called "Extreme"?
- What is level-wise splitting?
- What is leaf-wise splitting?
- Difference between level-wise and leaf-wise splitting.

- What is GOSS (Gradient-based One-Side Sampling)?
- What is EFB (Exclusive Feature Bundling)?
- How does XGBoost handle missing values?
- Why does XGBoost use second-order derivatives (Hessian)?
- What is the loss function in XGBoost?
- What is the regularization in XGBoost?
- What are the parameters of XGBoost?
- What is `n_estimators` in XGBoost?
- What is `learning_rate` in XGBoost?
- What is `max_depth` in XGBoost?
- What is `subsample` in XGBoost?
- What is `colsample_bytree` in XGBoost?
- What is `reg_alpha` in XGBoost?
- What is `reg_lambda` in XGBoost?
- What is `gamma` in XGBoost?
- What is `min_child_weight` in XGBoost?
- What is `early_stopping_rounds` ?
- What is `eval_metric` ?
- What is `verbosity` ?
- What is `n_jobs` ?
- What is `random_state` ?
- Why `random_state=42` ? Why 42 specifically?
- What is `n_estimators` in LightGBM?
- What is `num_leaves` in LightGBM?
- What is `learning_rate` in LightGBM?
- What is `subsample` in LightGBM?
- What is `colsample_bytree` in LightGBM?
- What is `reg_alpha` and `reg_lambda` in LightGBM?
- What is `subsample_freq` in LightGBM?
- What is CatBoost?
- How does CatBoost handle categorical features?

- What is ordered boosting in CatBoost?
- What is the difference between XGBoost, LightGBM, and CatBoost?
- When would you use XGBoost over LightGBM?
- When would you use LightGBM over XGBoost?
- When would you use CatBoost?
- Which model is best for high cardinality categorical features?
- Which model is fastest?
- Which model is most accurate?

Other Models

- What is KNN?
- How does KNN work?
- Why is KNN called a lazy learner?
- What is the K in KNN?
- How do you choose K?
- What happens if K is too small or too large?
- What is the curse of dimensionality in KNN?
- Is KNN sensitive to scaling?
- What is the difference between KNN and K-Means?
- What is SVM?
- How does SVM work?
- What is the kernel trick?
- What are the different types of kernels?
- What is the linear kernel?
- What is the polynomial kernel?
- What is the RBF kernel?
- What is the sigmoid kernel?
- What is the C parameter in SVM?
- What is the gamma parameter in SVM?
- What is the degree parameter in SVM?
- What is the difference between SVM and Logistic Regression?

- Which is better for outliers: SVM or Logistic Regression?
- What is the difference between SVC and LinearSVC?
- What is Naive Bayes?
- Why is it called "Naive"?
- What is the assumption of Naive Bayes?
- What are the types of Naive Bayes?
- What is the difference between generative and discriminative models?
- Is Logistic Regression generative or discriminative?
- Is Naive Bayes generative or discriminative?
- What is K-Means Clustering?
- How does K-Means work?
- What is the Elbow method?
- What is the difference between K-Means and K-Means++?
- What is the difference between K-Means and KNN?
- What is hierarchical clustering?
- What is DBSCAN?
- What is PCA?
- What is LDA?
- What is the difference between PCA and LDA?
- What is t-SNE?
- What is anomaly detection?
- What is the difference between classification and clustering?
- What is the difference between supervised and unsupervised learning?
- What is semi-supervised learning?
- What is reinforcement learning?
- What is the difference between machine learning and deep learning?
- What is a neural network?
- What are activation functions?
- What is ReLU?
- What is sigmoid?
- What is tanh?

- What is the input and output range of tanh?
- What is the input and output range of ReLU?
- What is the vanishing gradient problem?
- What is the exploding gradient problem?
- How many hidden layers can a neural network have?
- What happens if you increase the number of hidden layers?
- What is backpropagation?
- What is the difference between a perceptron and Logistic Regression?
- What is the difference between a perceptron and SVM?

5. MODEL EVALUATION

Metrics – Classification

- What is accuracy?
- Why is accuracy not a good metric for imbalanced data?
- What is precision?
- What is recall?
- What is F1 score?
- What is the formula for F1 score?
- What is the formula for precision?
- What is the formula for recall?
- What is the formula for accuracy?
- What is the difference between precision and recall?
- When do you use precision over recall?
- When do you use recall over precision?
- Give a real-life example where precision is more important than recall.
- Give a real-life example where recall is more important than precision.
- What is the F1 score if precision and recall are equal?
- What is macro average?
- What is weighted average?

- What is micro average?
- What is the difference between macro, micro, and weighted average?
- What is the "support" column in classification report?
- What is a confusion matrix?
- What are TP, TN, FP, FN?
- How do you identify TP, TN, FP, FN in a multi-class confusion matrix?
- What is Type I error?
- What is Type II error?
- What is ROC curve?
- What is AUC?
- What is ROC-AUC?
- What is the range of AUC?
- What does AUC = 0.5 mean?
- What does AUC = 1 mean?
- What is the difference between ROC curve and PR curve?
- When do you use PR curve over ROC curve?
- What is the difference between ROC-AUC and F1 score?
- Why did you plot ROC curve if you are already plotting F1 scores?
- What is the Gini coefficient?
- What is the Kolmogorov-Smirnov statistic?
- What is the lift curve?
- What is the gain curve?
- What is the cumulative response curve?

Metrics – Regression

- What is R^2 score?
- What is the formula for R^2 score?
- What is the range of R^2 score?
- Can R^2 be negative? Why?
- What is Adjusted R^2 ?
- What is the difference between R^2 and Adjusted R^2 ?

- What is MSE?
- What is RMSE?
- What is MAE?
- What is the difference between MSE and MAE?
- What is the difference between RMSE and MAE?
- What is the difference between MSE and RMSE?
- What is RMSLE?
- What is the difference between RMSE and RMSLE?
- Why is RMSLE used instead of RMSE?
- What is MAPE?
- What is SMAPE?
- What is Huber loss?
- When do you use MAE over MSE?
- Which metric is more robust to outliers?
- What is the formula for RMSE?
- What is the formula for RMSLE?
- What is the formula for R^2 score?

Model Validation

- What is train-test split?
- Why do we need train-test split?
- What is the train-test split ratio?
- Why 80-20? Why not 70-30?
- What is a validation set?
- Why do we need a validation set when we have train and test sets?
- What is cross-validation?
- What is K-Fold cross-validation?
- How does K-Fold cross-validation work?
- What is the advantage of K-Fold over train-test split?
- What is Stratified K-Fold?
- What is Time Series Split?

- What is Group K-Fold?
- When do you use Group K-Fold?
- What is Leave-One-Out cross-validation?
- What is the difference between cross-validation and train-test split?
- Is cross-validation used for training or evaluation?
- What is the bias-variance tradeoff?
- What is bias?
- What is variance?
- What is underfitting?
- What is overfitting?
- How do you detect overfitting?
- How do you detect underfitting?
- How do you prevent overfitting?
- How do you prevent underfitting?
- What is the relation between bias and underfitting?
- What is the relation between variance and overfitting?
- What happens to training and test scores in overfitting?
- What happens to training and test scores in underfitting?
- What is the ideal train-test score gap?
- What is the learning curve?
- What is the validation curve?

Hyperparameter Tuning

- What is hyperparameter tuning?
- Why do we need hyperparameter tuning?
- What are the different methods of hyperparameter tuning?
- What is GridSearchCV?
- How does GridSearchCV work?
- What is RandomizedSearchCV?
- How does RandomizedSearchCV work?
- What is the difference between GridSearchCV and RandomizedSearchCV?

- Which is faster: GridSearchCV or RandomizedSearchCV?
- Which is more thorough: GridSearchCV or RandomizedSearchCV?
- How many models will be trained in GridSearchCV?
- How many models will be trained in RandomizedSearchCV?
- What is `n_iter` in RandomizedSearchCV?
- What is `cv` in GridSearchCV?
- What is the default `cv` value?
- What happens if you increase `cv` ?
- What is `scoring` in GridSearchCV?
- What are the different scoring metrics?
- What is `n_jobs` in GridSearchCV?
- What is `verbose` in GridSearchCV?
- What is `refit` in GridSearchCV?
- What is `return_train_score` ?
- What is Bayesian optimization?
- What is Optuna?
- What is Hyperopt?
- What is manual hyperparameter tuning?
- How did you choose the hyperparameter grid?
- How did you decide the range of hyperparameters?
- How much time did hyperparameter tuning take?
- Did hyperparameter tuning improve your score?
- By how much did the score improve after HPT?
- What is the difference between parameters and hyperparameters?
- What is the difference between model parameters and hyperparameters?

6. STATISTICS & MATHEMATICS

- What is mean?
- What is median?
- What is mode?

- When do you use mean, median, and mode?
- What is the limitation of mean?
- What is variance?
- What is standard deviation?
- What is the difference between variance and standard deviation?
- What is the empirical rule (68-95-99.7 rule)?
- What is a normal distribution?
- What is skewness?
- What is kurtosis?
- What is the Central Limit Theorem?
- What is a p-value?
- What is the significance level (alpha)?
- What is the difference between p-value and alpha?
- What is a t-test?
- What is a chi-square test?
- What is ANOVA?
- What is the F-test?
- What is the difference between a t-test and ANOVA?
- What is correlation?
- What is covariance?
- What is the difference between correlation and covariance?
- What is the Pearson correlation coefficient?
- What is the Spearman correlation coefficient?
- What is the formula for Pearson correlation?
- What is the range of Pearson correlation?
- What is the difference between Pearson and Spearman correlation?
- What is the difference between correlation and causation?
- What is probability?
- What is likelihood?
- What is the difference between probability and likelihood?
- What is Bayes' theorem?

- What is conditional probability?
- What is the difference between independent and mutually exclusive events?
- What is a random variable?
- What is a probability distribution?
- What is a binomial distribution?
- What is a Poisson distribution?
- What is a uniform distribution?
- What is an exponential distribution?
- What is the difference between discrete and continuous distributions?
- What is the law of large numbers?
- What is the difference between a statistic and a parameter?
- What is sampling?
- What is sampling bias?
- What is the difference between population and sample?
- What is standard error?
- What is the confidence interval?
- What is the margin of error?
- What is hypothesis testing?
- What is the null hypothesis?
- What is the alternative hypothesis?
- What is a Type I error?
- What is a Type II error?
- What is the power of a test?
- What is the difference between a one-tailed and two-tailed test?
- What is a Z-score?
- What is the difference between a Z-test and a t-test?
- What is the difference between a parametric and non-parametric test?
- What is the Mann-Whitney U test?
- What is the Wilcoxon signed-rank test?
- What is the Kruskal-Wallis test?
- What is the Friedman test?

7. PYTHON & CODING

Pandas

- How do you load a dataset in pandas?
- How do you display the first 5 rows?
- How do you display the last 5 rows?
- How do you display the shape of a DataFrame?
- How do you display the column names?
- How do you display the data types of columns?
- How do you display the summary statistics?
- How do you check for null values?
- How do you count null values in each column?
- How do you drop a column?
- How do you drop rows with null values?
- How do you fill null values?
- How do you filter rows based on a condition?
- How do you filter rows where a column value is greater than a threshold?
- How do you filter rows where a column value is in a list?
- How do you select only numerical columns?
- How do you select only categorical columns?
- How do you convert a column to a different data type?
- How do you apply a function to a column?
- How do you group by a column and aggregate?
- How do you sort a DataFrame?
- How do you reset the index?
- How do you rename columns?
- How do you concatenate DataFrames?
- How do you merge DataFrames?
- How do you handle duplicate rows?
- How do you find unique values in a column?

- How do you count the frequency of each unique value?
- How do you replace values in a column?
- How do you replace '?' with NaN?
- How do you create a new column?
- How do you drop rows based on a condition?
- How do you select rows by index?
- How do you select rows by label?
- How do you use `iloc` ?
- How do you use `loc` ?
- How do you use `groupby` ?
- How do you use `pivot_table` ?
- How do you use `crosstab` ?
- How do you use `value_counts()` ?
- How do you use `describe()` ?
- How do you use `info()` ?
- How do you use `isnull()` ?
- How do you use `notnull()` ?
- How do you use `dropna()` ?
- How do you use `fillna()` ?
- How do you use `astype()` ?
- How do you use `apply()` ?
- How do you use `map()` ?
- How do you use `replace()` ?
- How do you use `sort_values()` ?
- How do you use `reset_index()` ?
- How do you use `rename()` ?
- How do you use `concat()` ?
- How do you use `merge()` ?
- How do you use `join()` ?

NumPy

- How do you create a NumPy array?
- How do you create an array of zeros?
- How do you create an array of ones?
- How do you create an array with a range of values?
- How do you create a random array?
- How do you create a random matrix?
- How do you find the shape of an array?
- How do you find the dimensions of an array?
- How do you find the data type of an array?
- How do you index a NumPy array?
- How do you slice a NumPy array?
- How do you select a column from a NumPy array?
- How do you select a row from a NumPy array?
- How do you perform element-wise operations?
- How do you perform matrix multiplication?
- How do you find the mean, median, and standard deviation?
- How do you find the minimum and maximum?
- How do you find the sum and product?
- How do you transpose an array?
- How do you reshape an array?
- How do you concatenate arrays?
- How do you stack arrays?
- What is `hstack` ?
- What is `vstack` ?
- How do you find the correlation matrix?
- How do you find the covariance matrix?
- How do you find the eigenvalues and eigenvectors?
- How do you solve a system of linear equations?
- How do you generate random numbers with a seed?
- How do you set a random seed?
- How do you create a boolean mask?

- How do you use `np.where()` ?
- How do you use `np.select()` ?
- How do you use `np.clip()` ?
- How do you use `np.log()` ?
- How do you use `np.log1p()` ?
- How do you use `np.exp()` ?
- How do you use `np.sqrt()` ?
- How do you use `np.power()` ?
- How do you use `np.abs()` ?
- How do you use `np.round()` ?
- How do you use `np.floor()` ?
- How do you use `np.ceil()` ?

Scikit-learn

- How do you import a dataset from sklearn?
- How do you load the Iris dataset?
- How do you load the Diabetes dataset?
- How do you load the Breast Cancer dataset?
- How do you load the California Housing dataset?
- How do you load the Digits dataset?
- How do you load the Wine dataset?
- How do you split data into train and test sets?
- How do you use `train_test_split` ?
- What is `random_state` in `train_test_split` ?
- What is `stratify` in `train_test_split` ?
- What is `shuffle` in `train_test_split` ?
- How do you create a pipeline?
- How do you create a `ColumnTransformer`?
- How do you create a `SimpleImputer`?
- How do you create a `StandardScaler`?
- How do you create a `MinMaxScaler`?

- How do you create a RobustScaler?
- How do you create a OneHotEncoder?
- How do you create a LabelEncoder?
- How do you create an OrdinalEncoder?
- How do you create a TargetEncoder?
- How do you create a LinearRegression model?
- How do you create a LogisticRegression model?
- How do you create a DecisionTreeClassifier?
- How do you create a RandomForestClassifier?
- How do you create a RandomForestRegressor?
- How do you create an XGBoost model?
- How do you create a LightGBM model?
- How do you create a CatBoost model?
- How do you create an SVM model?
- How do you create a KNN model?
- How do you create a K-Means model?
- How do you create a PCA model?
- How do you create a GridSearchCV?
- How do you create a RandomizedSearchCV?
- How do you fit a model?
- How do you predict using a model?
- How do you evaluate a model?
- How do you calculate accuracy?
- How do you calculate precision?
- How do you calculate recall?
- How do you calculate F1 score?
- How do you calculate R^2 score?
- How do you calculate MSE?
- How do you calculate RMSE?
- How do you calculate MAE?
- How do you create a confusion matrix?

- How do you create a classification report?
- How do you plot an ROC curve?
- How do you plot a precision-recall curve?
- How do you save a model?
- How do you load a model?
- How do you use `joblib` ?
- How do you use `pickle` ?

Matplotlib & Seaborn

- How do you import matplotlib?
- How do you import seaborn?
- How do you create a line plot?
- How do you create a bar plot?
- How do you create a histogram?
- How do you create a scatter plot?
- How do you create a box plot?
- How do you create a violin plot?
- How do you create a count plot?
- How do you create a heatmap?
- How do you create a pair plot?
- How do you create a dist plot?
- How do you create a joint plot?
- How do you create a subplot?
- How do you add a title?
- How do you add axis labels?
- How do you add a legend?
- How do you change the figure size?
- How do you rotate x-axis labels?
- How do you set axis limits?
- How do you add annotations?
- How do you save a plot?

- What is the difference between matplotlib and seaborn?
- Why do you use both matplotlib and seaborn?
- How do you plot a correlation matrix?
- How do you plot a confusion matrix?
- How do you plot feature importance?
- How do you plot a learning curve?
- How do you plot a validation curve?
- How do you plot an elbow curve?
- How do you plot clusters?

Regex

- What is regex?
- How do you use regex in pandas?
- How do you extract patterns using regex?
- How do you replace patterns using regex?
- What does `re.sub()` do?
- What does `re.findall()` do?
- What does `re.search()` do?
- What does `re.match()` do?
- What does `re.compile()` do?
- What does `\d` mean in regex?
- What does `\w` mean in regex?
- What does `\s` mean in regex?
- What does `.` mean in regex?
- What does `*` mean in regex?
- What does `+` mean in regex?
- What does `?` mean in regex?
- What does `[]` mean in regex?
- What does `()` mean in regex?
- What does `|` mean in regex?
- What does `^` mean in regex?

- What does `$` mean in regex?
- How do you remove punctuation using regex?
- How do you remove numbers using regex?
- How do you remove special characters using regex?
- How do you extract email addresses using regex?
- How do you extract URLs using regex?

General Coding

- Write a Python program to find the GCD of two numbers.
- Write a Python program for the Fibonacci series.
- Write a Python program to check if a number is prime.
- Write a Python program to reverse a string.
- Write a Python program to check if a string is a palindrome.
- Write a Python program to find the factorial of a number.
- Write a Python program to sort a list.
- Write a Python program to find the maximum and minimum in a list.
- Write a Python program to remove duplicates from a list.
- Write a Python program to count the frequency of elements in a list.
- Write a Python program to find the second largest number in a list.
- Write a Python program to merge two dictionaries.
- Write a Python program to find common elements in two lists.
- Write a Python program to transpose a matrix.
- Write a Python program to multiply two matrices.
- Write a Python program to implement `train_test_split` from scratch.
- Write a Python program to implement `SimpleImputer` from scratch.
- Write a Python program to implement `StandardScaler` from scratch.
- Write a Python program to implement `OneHotEncoder` from scratch.
- Write a Python program to implement Logistic Regression from scratch using NumPy.
- Write a Python program to implement Linear Regression from scratch using NumPy.
- Write a Python program to implement K-Means from scratch.
- Write a Python program to implement KNN from scratch.

- Write a Python program to implement PCA from scratch.
- Write a Python program to implement Gradient Descent from scratch.

Live Coding Tasks (Common)

- Load the Iris dataset and perform train-test split.
- Load the Diabetes dataset and print the feature matrix.
- Load the Breast Cancer dataset and print the target names.
- Load the California Housing dataset and print the feature names.
- Load the Digits dataset and print the target values.
- Load the Wine dataset and print the top 5 features.
- Create a pipeline with StandardScaler and OneHotEncoder.
- Create a pipeline with SimpleImputer and Logistic Regression.
- Create a pipeline with RobustScaler and SVM.
- Perform RFE on a dataset to select top 2 features.
- Perform PCA on a dataset and show the variance.
- Perform train-test split and print the shapes.
- Perform OneHotEncoding on a categorical column.
- Perform SimpleImputer with `strategy='most_frequent'` .
- Perform SimpleImputer with `strategy='mean'` .
- Plot a histogram of the target variable.
- Plot a bar graph of model comparison.
- Plot a correlation heatmap.
- Plot a box plot.
- Plot a scatter plot.
- Filter a DataFrame based on a condition.
- Print the unique values in a column.
- Print the frequency of each class in the target column.
- Replace '?' with NaN in a DataFrame.
- Convert a column to numeric data type.
- Drop a column from a DataFrame.
- Create a new DataFrame with specific columns.

- Merge/concatenate two DataFrames.
- Handle missing values in a DataFrame.
- Encode categorical variables in a DataFrame.
- Scale numerical variables in a DataFrame.
- Fit a Logistic Regression model and print the accuracy.
- Fit a Random Forest model and print the classification report.
- Fit an SVM model and print the confusion matrix.
- Fit a Decision Tree model and plot the tree.
- Fit a K-Means model and plot the clusters.
- Fit a Linear Regression model and print the R^2 score.
- Fit a Ridge/Lasso model and print the coefficients.
- Implement GridSearchCV for a model.
- Implement RandomizedSearchCV for a model.
- Implement cross-validation for a model.
- Implement RFE for feature selection.
- Implement PCA for dimensionality reduction.
- Implement TF-IDF on a corpus.
- Implement CountVectorizer on a corpus.
- Implement a custom train-test split function.
- Implement a custom confusion matrix function.
- Implement precision, recall, and F1 score from scratch.
- Implement a custom imputer function.
- Implement a custom scaler function.
- Create a synthetic dataset and fit a model.
- Create a random dataset with specific dimensions.
- Append/stack datasets using `hstack` / `vstack` .
- Double-stack a dataset.
- Create a DataFrame from a dictionary.
- Create a DataFrame with specific column names.
- Create a DataFrame with specific values.
- Create a DataFrame with missing values.

- Create a DataFrame with categorical and numerical columns.
- Create a DataFrame with a target column.

8. DEPLOYMENT & PRODUCTION

- How would you deploy this model?
- What is the process of deploying a model?
- What is FastAPI?
- What is Uvicorn?
- What is Flask?
- What is Django?
- What is Docker?
- What is Kubernetes?
- What is the difference between Docker and Kubernetes?
- What is CI/CD?
- What is MLOps?
- What is model monitoring?
- What is model drift?
- What is data drift?
- What is concept drift?
- How do you handle model drift?
- How do you handle multiple concurrent requests?
- How do you scale a model in production?
- How do you optimize a model for production?
- What is model quantization?
- What is model pruning?
- What is model distillation?
- What is ONNX?
- What is TensorFlow Serving?
- What is TorchServe?
- What is MLflow?

- What is Kubeflow?
- What is Airflow?
- What is a feature store?
- What is a model registry?
- What is A/B testing?
- What is canary deployment?
- What is blue-green deployment?
- What is shadow deployment?
- What is the difference between batch and real-time inference?
- What is the difference between online and offline learning?
- How do you handle large datasets in production?
- How do you handle 500GB of data on a laptop?
- How do you chunk data for parallel processing?
- What is the role of caching in production?
- What is Redis?
- What is Celery?
- What is a message queue?
- What is Kafka?
- What is RabbitMQ?
- What is the difference between Redis and a database?
- What is the difference between SQL and NoSQL?
- What is the difference between OLTP and OLAP?
- What is a data warehouse?
- What is a data lake?
- What is a data lakehouse?
- What is ETL?
- What is ELT?
- What is a data pipeline?
- What is Apache Spark?
- What is Hadoop?
- What is MapReduce?

- What is HDFS?
- What is Apache Hive?
- What is Apache Pig?
- What is Apache HBase?
- What is Apache Cassandra?
- What is MongoDB?
- What is PostgreSQL?
- What is MySQL?
- What is SQLite?

9. GENERAL & HR QUESTIONS

- Introduce yourself.
- Tell me about yourself.
- What is your academic background?
- What are you currently doing?
- Why did you choose this course?
- What do you want to do after this course?
- What are your future plans?
- Are you planning to do a master's?
- Are you planning to do a job?
- Are you participating in any hackathons?
- What are your hobbies?
- What are your strengths?
- What are your weaknesses?
- What did you learn from this project?
- What was the most challenging part of this project?
- How did you overcome those challenges?
- What did you enjoy the most in this project?
- What would you do differently if you had more time?
- How would you improve this project?

- How would you explain this project to a non-technical person?
- How would you explain this project to a business person?
- What are the business implications of this project?
- How can this project be used in the real world?
- Who will use your solution?
- What are the stakeholders of this project?
- How would you measure the success of this project?
- What is the ROI of this project?
- What are the limitations of your project?
- What are the ethical considerations of your project?
- What are the privacy concerns of your project?
- What are the biases in your project?
- How do you ensure fairness in your model?
- How do you ensure reproducibility?
- How do you ensure data security?
- How do you ensure model interpretability?
- How do you ensure model explainability?
- What is explainable AI?
- What is LIME?
- What is SHAP?
- What is feature importance?
- What is permutation importance?
- What is partial dependence plot?
- What is ICE plot?
- What is a waterfall plot?
- What is a force plot?
- What is a summary plot?
- What is a dependence plot?
- What is a decision plot?

10. PROCTOR-SPECIFIC PATTERNS

Proctor level1_118

- Very chill, mostly asks to scroll through notebook.
- Questions on EDA graphs, correlation, parametric/non-parametric models.
- Coding: load Iris/Breast Cancer/Diabetes, plot histogram, basic model fitting.
- Theory: bagging vs boosting, weak learners, L1/L2, parametric models.
- No deep theory, focuses on notebook and basic coding.

Proctor mlp_level1_viva7

- Very chill, asks to explain notebook in 15-25 mins.
- Questions on GridSearchCV vs RandomizedSearchCV, OHE vs Label Encoding.
- Coding: print columns, unique values, replace values, SimpleImputer.
- No deep theory, focuses on notebook explanation.

Proctor mlp_level1_viva9

- Sweet and supportive, asks to explain notebook.
- Questions on R^2 score, bagging/boosting, bias-variance tradeoff.
- Coding: load data, train-test split, pipeline, SimpleImputer, filtering.
- Focuses on EDA, preprocessing, HPT.

Proctor mlp_level1_viva13

- Checks rubrics, asks to scroll notebook.
- Questions on box plot (full name), n_jobs, reg_alpha, reg_lambda, subsample, colsample_bytree.
- Coding: train_test_split, plot graph, filter data.
- Focuses on graphs and HPT parameters.

Proctor mlp_level1_viva14

- Very chill, asks to explain notebook in detail.
- Questions on CV, learning rate, overfitting/underfitting, pipeline.

- Coding: load data, filter, basic model fitting.
- Focuses on notebook explanation and basic theory.

Proctor level2_74

- Very repetitive, asks same questions to everyone.
- Questions on data preprocessing, feature scaling, hyperparameter tuning, random_state=42, dimensionality reduction, house price prediction.
- No coding, focuses on theory.
- Prepare all questions from this sheet for this proctor.

Proctor level2_81

- Very thorough, asks deep questions.
- Questions on EDA insights, encoding, scaling, TF-IDF, model working, L1/L2, PCA, Naive Bayes, SVM, SGD, Logistic Regression formulas, RFE.
- Coding: RFE on dataset, word cloud, train model with GridSearchCV.
- Focuses on EDA and model theory.

Proctor level2_106

- Very chill, asks to explain notebook in detail.
- Questions on loss function of Logistic Regression, HPT, pipeline, F1 score, overfitting, time for HPT, best model.
- No coding, focuses on notebook explanation.

Proctor level2_107

- Very thorough, asks deep conceptual questions.
- Questions on imputation, outliers, scaling, Logistic Regression, SVM, Decision Trees, information gain, entropy, Random Forest, precision, recall, bagging, boosting, HPT, MNIST coding, neural networks, activation functions, tanh, step function, sigmoid, loss function, KNN, oversampling/undersampling.
- Coding: MNIST with pipeline, SVM, F1 score.
- Focuses on deep theory and coding.

Proctor level2_123

- Chill, asks to explain notebook in detail.
- Questions on learning rate, n-grams, stopwords, libraries, overfitting, missing values, OHE, SimpleImputer.
- Coding: train_test_split, basic model fitting.
- Focuses on notebook explanation.

Proctor level2_124

- Very chill, gives dataset link for coding.
- Questions on RFE, filtering, correlation, normalization, hyperparameter tuning, GridSearch vs RandomizedSearch, preprocessing, Logistic Regression naming, KNN lazy learner, Decision Trees.
- Coding: load dataset, filter rows, RFE, correlation, normalization.
- Focuses on coding and basic theory.

Proctor level3_9

- Asks many theory questions.
- Questions on preprocessing, EDA, dimensionality reduction, loss function, overfitting/underfitting, Linear Regression, bagging/boosting, scaling, PCA, best model, K-Means, KNN, Logistic Regression, Decision Tree, confusion matrix, ROC curve.
- No coding, focuses on theory.

Proctor level3_20

- Very chill, asks to explain notebook.
- Questions on class imbalance, PCA, feature engineering, HPT, leaderboard score, model comparison.
- No coding, focuses on notebook explanation.
- Gives suggestions for improvement.

Proctor level3_29

- Very thorough, asks deep theory and coding.

- Questions on ML types, supervised/unsupervised, reinforcement learning, K-Means, K-Means++, Decision Tree, Gini, entropy, information gain, bagging/boosting, SVM, PCA, neural networks, activation functions, overfitting/underfitting, regularization, RMSLE, Random Forest vs XGBoost, Ridge loss function, MSE, target variable, ML vs DL.
- Coding: random dataset, Random Forest, PCA, train-test split, plot accuracy vs components, Iris dataset, Decision Tree, plot tree, K-Means, optimal clusters, elbow plot, SVM.
- Focuses on deep theory and coding.

Proctor level3_31

- Very sweet and supportive.
- Questions on self-intro, notebook explanation, mean/median/mode, HPT, EDA, pipeline, feature engineering, model explanation, oversampling, loss function, future plans.
- No coding, focuses on notebook and career.
- Easier than Level 1.

Proctor level3_44

- Very nice and supportive.
- Questions on problem statement, notebook explanation, feature engineering, pipeline, EDA insights, scaling, encoding, missing values, model selection, HPT, real-world application, R^2 formula, regularization, ensemble models.
- Coding: imputation of numerical and categorical data.
- Focuses on notebook explanation and real-world application.

Proctor proctor776

- Very calm and knowledgeable.
- Questions on notebook explanation, bias-variance, GOSS, XGBoost missing values, Hessian loss, bagging vs boosting, model improvement, stakeholders, deployment, train-test split from scratch.
- Coding: train_test_split from scratch.
- Focuses on deep theory and deployment.

Proctor proctor779

- Very encouraging and friendly.
- Questions on notebook explanation, EDA graphs, feature engineering, split method, encoding, missing values, hyperparameters, model parameters.
- Coding: mode of numerical/categorical values, frequency of categorical column.
- Focuses on notebook explanation, no theory if explained well.

Proctor proctor782

- Very strict and thorough.
- Questions on every part of notebook, EDA, feature engineering, preprocessing, models, HPT, parameters, log transformation, outliers, OHE, learning rate, gradient descent, validation sets, bagging/boosting.
- Coding: histogram plot from scratch, modify histogram.
- Very long viva (1.5 hours), deep follow-up questions.

Proctor proctor784

- Very strict, deep theory.
- Questions on evaluation metrics, RMSLE vs RMSE, log vs log1p, gradient descent, SGD, bagging/boosting, precision/recall, class imbalance, overfitting/underfitting, regularization, dimensionality reduction, PCA, loss function, RMSLE formula, train/test/validation split, cardinality, OHE, activation functions.
- No coding, focuses on deep theory.
- Expects knowledge of every word in notebook.

Proctor proctor786

- Very thorough, asks many questions.
- Questions on box plot, median, Pearson correlation, target encoding, standard deviation, GCD coding, MSE vs RMSE, RMSLE, IQR, mean/median/variance, naive target encoding, Type I error, L1/L2, XGBoost, overfitting prevention, boosting, leaves, early stopping, entropy, Gini, mean/median/mode, limitations of mean.
- Coding: GCD function.
- Focuses on deep conceptual questions.

Proctor proctor787

- Chill, asks about EDA, charts, models, internal working.
- Questions on charts, insights, model working, gradient boosting, decision trees, feature selection, splits, ensemble, bagging, boosting.
- Focuses on EDA and model understanding.

Proctor proctor788

- Friendly and patient.
- Questions on encoders, target encoder, column selection, final model, Ridge Regression, Linear Regression.
- No coding, feels like a conversation.

Proctor proctor790

- Chill, asks about PCA, confusion matrix, regression/classification, supervised/unsupervised, log transformation.
- Gives suggestions, focuses on notebook.

Proctor proctor601

- Patient and not strict.
- Questions on missing values, preprocessing, target encoding vs label encoding, GridSearch vs RandomizedSearch, model comparison.
- Coding: load dataset, show columns, null values, last 5 rows, range of transaction year.

Proctor proctor610

- Very professional and helpful.
- Questions on classification vs regression, scaling, tree-based models, imputers, encoders, pipelines, GroupKFold, data leakage, fit_transform vs transform, challenges faced.
- Coding: load Iris, SimpleImputer, train-test split.

Proctor proctor746

- Chill, asks about notebook, skewness, correlation, overfitting, variance-bias tradeoff, K-Fold, imputation.
- Questions on Naive Bayes, bagging/boosting, perceptron vs Logistic Regression vs SVM, sigmoid, regularization, underfitting/overfitting.
- Focuses on theory and notebook.

Proctor proctor727

- Chill but deep.
- Questions on notebook explanation, coding with manual DataFrame, imports, train-test split, missing values, median imputation, encoding, OHE, scaling.
- Questions on model working (Decision Trees, Random Forest, bagging, boosting, XGBoost, LightGBM, CatBoost), hyperparameters, overfitting/underfitting.
- Coding: everything from scratch with manual DataFrame.

Proctor proctor785

- Sadistic behavior, not helpful.
- Questions on notebook, tries to confuse, contradicts own statements.
- Expects you to be a data scientist already.
- Stay calm, answer confidently.

Proctor proctor774

- Very chill.
- Questions on bagging/boosting, Logistic Regression, L1/L2, precision/recall, accuracy, ROC curve, RMSLE, Decision Tree, entropy, ensemble methods.
- Focuses on clear, flowy answers.

Proctor level3_28

- Very sweet, focuses on EDA and visuals.
- Questions on academic background, problem statement, notebook explanation, insights from graphs, correlation, models, model comparison, code explanation.
- No theory beyond notebook.
- Good graphs impress her.

Proctor level3_30

- Theory heavy, no coding.
- Questions on imbalanced data, Pearson correlation formula, bagging classifiers, PCA linear/non-linear, L1/L2, entropy, information gain, K-Means, KNN, SVM, PCA, neural networks, activation functions.
- Focuses on theory.

Proctor level3_32

- Theory and visualization heavy.
- Questions on descriptive statistics, label vs word count, correlation matrix, Logistic Regression, weights, class_weight, feature engineering, confusion matrix, metrics, TF-IDF, model explanation.
- Coding: label vs most frequent words, confusion matrix code.
- Focuses on theory and graphs.

Proctor level3_46

- Nice and nurturing.
- Questions on outliers, collinearity, bagging/boosting, Decision Tree, entropy, metrics, accuracy, diabetes metric, Logistic Regression, KNN.
- Focuses on notebook explanation.

11. COMMON THEMES & TIPS

What Most Proctors Ask

1. **Notebook explanation** – Explain your notebook in detail, covering EDA, preprocessing, feature engineering, models, HPT, and results.
2. **Why questions** – Why did you use this method? Why not another? Why this parameter?
3. **EDA insights** – What did you learn from your graphs? What patterns did you find?
4. **Model working** – How does each model you used work? What are its parameters?
5. **Preprocessing** – Imputation, scaling, encoding, pipelines, data leakage.

6. **Hyperparameter tuning** – GridSearch vs RandomizedSearch, parameters, time taken.
7. **Overfitting/Underfitting** – How to detect, how to prevent, bias-variance tradeoff.
8. **Evaluation metrics** – Precision, recall, F1, accuracy, ROC-AUC, R^2 , RMSE, RMSLE.
9. **Bagging vs Boosting** – Difference, examples, when to use which.
10. **Coding** – Basic data loading, filtering, train-test split, pipeline, model fitting, imputation, encoding, plotting.

General Tips

- **Know your notebook inside out** – Every line, every parameter, every decision.
- **Be able to justify every choice** – Why this method? Why not another?
- **Practice explaining your notebook** – Aim for 20-25 minutes of detailed explanation.
- **Prepare theory** – MLT, MLF, MLP concepts, especially models you used.
- **Practice coding** – Basic pandas, numpy, sklearn, matplotlib, seaborn.
- **Be confident** – If you don't know, say so honestly.
- **Use documentation** – Allowed in most vivas for coding questions.
- **Don't use LLMs** – Most proctors will deduct marks if you use LLMs during coding.
- **Keep your notebook clean** – Well-commented, well-organized, with markdown cells.
- **Add graphs** – Model comparison graphs, EDA graphs, confusion matrix.
- **Know your EDA** – Insights from each graph, why you plotted it.
- **Know your models** – How they work, parameters, pros/cons, why they performed well/poorly.
- **Know your preprocessing** – Why you imputed, scaled, encoded the way you did.
- **Know your HPT** – Parameters, ranges, time taken, improvement in score.
- **Know your evaluation** – Why you chose the metric you did, what it means.
- **Know your data** – Columns, types, distributions, correlations, outliers, missing values.
- **Know your problem statement** – Classification/regression, supervised/unsupervised, objective.
- **Know your results** – Best model, score, leaderboard rank, why it performed best.
- **Know your limitations** – What could be improved, what you would do differently.
- **Know your future scope** – How you would deploy, how you would improve, real-world application.

Proctor-Specific Tips

- **For chill proctors** (118, viva7, viva14, viva9, level2_106, level3_20, proctor779, proctor774): Explain your notebook in detail, take your time, and you won't get many questions.
- **For strict proctors** (level2_81, level2_107, proctor782, proctor784, proctor786, proctor785): Know everything deeply, prepare theory thoroughly, be ready for cross-questions and coding.
- **For repetitive proctors** (level2_74, level2_124): Prepare all questions from this sheet, they ask the same questions to everyone.
- **For theory-heavy proctors** (level3_9, level3_29, level3_30, level3_32): Revise MLT, MLF, MLP theory, formulas, model working, statistics.
- **For coding-heavy proctors** (level2_103, level2_107, level2_124, level3_29): Practice coding, especially RFE, PCA, pipelines, train-test split, model fitting, plotting.

12. FORMULAS TO REMEMBER

Classification Metrics

- **Accuracy** = $(TP + TN) / (TP + TN + FP + FN)$
- **Precision** = $TP / (TP + FP)$
- **Recall (Sensitivity)** = $TP / (TP + FN)$
- **Specificity** = $TN / (TN + FP)$
- **F1 Score** = $2 \times (Precision \times Recall) / (Precision + Recall)$
- **False Positive Rate (FPR)** = $FP / (FP + TN)$
- **True Positive Rate (TPR)** = $TP / (TP + FN)$
- **Macro Average** = Average of metric across all classes (unweighted)
- **Weighted Average** = Weighted average of metric by support
- **Micro Average** = Global average (sum TP, FP, FN across classes)

Regression Metrics

- **MSE** = $(1/n) \times \sum (y_i - \hat{y}_i)^2$
- **RMSE** = $\sqrt{MSE}$

- **MAE** = $(1/n) \times \sum |y_i - \hat{y}_i|$
- **R²** = $1 - (SS_{\text{res}} / SS_{\text{tot}})$
- **Adjusted R²** = $1 - [(1 - R^2)(n - 1) / (n - k - 1)]$
- **RMSLE** = $\sqrt{[(1/n) \times \sum (\log(y_i + 1) - \log(\hat{y}_i + 1))^2]}$
- **MAPE** = $(1/n) \times \sum |(y_i - \hat{y}_i) / y_i| \times 100$
- **SMAPE** = $(1/n) \times \sum |y_i - \hat{y}_i| / ((|y_i| + |\hat{y}_i|) / 2) \times 100$

Statistics

- **Mean** = $\sum x_i / n$
- **Variance** = $\sum (x_i - \mu)^2 / n$
- **Standard Deviation** = $\sqrt{\text{Variance}}$
- **Covariance** = $\sum (x_i - \bar{x})(y_i - \bar{y}) / n$
- **Pearson Correlation** = $\text{Cov}(x, y) / (\sigma_x \times \sigma_y)$
- **Z-score** = $(x - \mu) / \sigma$
- **IQR** = $Q3 - Q1$
- **Lower Bound** = $Q1 - 1.5 \times \text{IQR}$
- **Upper Bound** = $Q3 + 1.5 \times \text{IQR}$
- **Sigmoid** = $1 / (1 + e^{-z})$
- **Tanh** = $(e^z - e^{-z}) / (e^z + e^{-z})$
- **ReLU** = $\max(0, z)$
- **Entropy** = $-\sum p_i \times \log_2(p_i)$
- **Gini Index** = $1 - \sum p_i^2$
- **Information Gain** = $\text{Entropy}(\text{parent}) - \sum (\text{weighted Entropy}(\text{children}))$

Regularization

- **L1 (Lasso)** = $\text{Loss} + \lambda \times \sum |w_i|$
- **L2 (Ridge)** = $\text{Loss} + \lambda \times \sum w_i^2$
- **ElasticNet** = $\text{Loss} + \lambda_1 \times \sum |w_i| + \lambda_2 \times \sum w_i^2$

PCA

- **Covariance Matrix** = $(1/n) \times X^T X$

- **Eigenvalues and Eigenvectors** = $\text{Cov} \times v = \lambda \times v$
- **Explained Variance Ratio** = $\lambda_i / \sum \lambda_j$
- **Principal Components** = $X \times \text{Eigenvectors}$

Gradient Descent

- **Weight Update** = $w = w - \eta \times \partial \text{Loss} / \partial w$
- **Learning Rate (η)** = Step size
- **Batch GD** = Uses all samples
- **Stochastic GD** = Uses one sample
- **Mini-Batch GD** = Uses a batch of samples

13. CODING CHEAT SHEET

Load Data

python

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris, load_diabetes, load_breast_cancer, load_digits, load_wine, fetch_california_housing

# Load from sklearn
iris = load_iris()
X = iris.data
y = iris.target

# Load from CSV
df = pd.read_csv('train.csv')
```

Basic DataFrame Operations

python

```
df.head()
df.tail()
df.shape
df.columns
```

```

df.dtypes
df.info()
df.describe()
df.isnull().sum()
df.drop(columns=['col'])
df.dropna()
df.fillna(0)
df['col'].value_counts()
df['col'].unique()
df['col'].nunique()
df[df['col'] > 50]
df[(df['col1'] == 'A') & (df['col2'] > 50)]
df.loc[:, 'col']
df.iloc[:, 0]
df['new_col'] = df['col1'] + df['col2']
df.rename(columns={'old': 'new'})
df.sort_values(by='col')
df.reset_index(drop=True)
pd.concat([df1, df2])
pd.merge(df1, df2, on='col')

```

Preprocessing

python

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler, OneHotEncoder, LabelEncoder, OrdinalEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Imputation
imputer = SimpleImputer(strategy='mean') # or 'median', 'most_frequent', 'constant'
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)

# Scaling
scaler = StandardScaler() # or MinMaxScaler(), RobustScaler()
X_train_scaled = scaler.fit_transform(X_train_imputed)
X_test_scaled = scaler.transform(X_test_imputed)

# Encoding

```

```

ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore', drop='first')
X_train_encoded = ohe.fit_transform(X_train_cat)
X_test_encoded = ohe.transform(X_test_cat)

le = LabelEncoder()
y_encoded = le.fit_transform(y)

oe = OrdinalEncoder()
X_train_ord = oe.fit_transform(X_train_cat)

# Pipeline
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(sparse_output=False, handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_cols),
    ('cat', cat_pipeline, cat_cols)
])

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

```

Models

python

```

from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge, Lasso, ElasticNet, SGDClassifier, SGDRegressor
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor, GradientBoostingClassifier, GradientBoostingRegressor, ExtraTreesClassifier, ExtraTreesRegressor, VotingClassifier, StackingClassifier
from sklearn.svm import SVC, SVR, LinearSVC
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA, TruncatedSVD
from xgboost import XGBClassifier, XGBRegressor
from lightgbm import LGBMClassifier, LGBMRegressor
from catboost import CatBoostClassifier, CatBoostRegressor

```

```
# Linear Regression
```

```
lr = LinearRegression()  
lr.fit(X_train, y_train)  
y_pred = lr.predict(X_test)
```

```
# Logistic Regression
```

```
log_reg = LogisticRegression(C=1.0, max_iter=1000, solver='lbfgs', class_weight='balanced')  
log_reg.fit(X_train, y_train)  
y_pred = log_reg.predict(X_test)
```

```
# Random Forest
```

```
rf = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42, n_jobs=-1)  
rf.fit(X_train, y_train)  
y_pred = rf.predict(X_test)
```

```
# XGBoost
```

```
xgb = XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=5, subsample=0.8, colsample_bytree=0.8, reg_alpha=0.1, reg_lambda=1.0, random_state=42)  
xgb.fit(X_train, y_train)  
y_pred = xgb.predict(X_test)
```

```
# LightGBM
```

```
lgbm = LGBMClassifier(n_estimators=100, learning_rate=0.1, num_leaves=31, subsample=0.8, colsample_bytree=0.8, reg_alpha=0.1, reg_lambda=1.0, random_state=42)  
lgbm.fit(X_train, y_train)  
y_pred = lgbm.predict(X_test)
```

```
# SVM
```

```
svm = SVC(kernel='rbf', C=1.0, gamma='scale')  
svm.fit(X_train, y_train)  
y_pred = svm.predict(X_test)
```

```
# KNN
```

```
knn = KNeighborsClassifier(n_neighbors=5)  
knn.fit(X_train, y_train)  
y_pred = knn.predict(X_test)
```

```
# K-Means
```

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)  
kmeans.fit(X)  
labels = kmeans.labels_
```

```
# PCA
```

```
pca = PCA(n_components=2)  
X_pca = pca.fit_transform(X)
```

Evaluation

python

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report, roc_auc_score, roc_curve, mean_squared_error, mean_absolute_error, r2_score, mean_squared_log_error
```

Classification

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro')
recall = recall_score(y_test, y_pred, average='macro')
f1 = f1_score(y_test, y_pred, average='macro')
cm = confusion_matrix(y_test, y_pred)
cr = classification_report(y_test, y_pred)
auc = roc_auc_score(y_test, y_pred)
```

Regression

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
rmsle = np.sqrt(mean_squared_log_error(y_test, y_pred))
```

Hyperparameter Tuning

python

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV, cross_val_score, KFold, StratifiedKFold
```

GridSearchCV

```
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10]
}
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid, cv=5, scoring='f1_macro', n_jobs=-1, verbose=1)
grid_search.fit(X_train, y_train)
best_params = grid_search.best_params_
best_score = grid_search.best_score_
```

RandomizedSearchCV

```
param_dist = {
    'n_estimators': [100, 200, 300, 400, 500],
    'max_depth': [5, 10, 15, 20, None],
```

```

    'min_samples_split': [2, 5, 10, 15],
    'min_samples_leaf': [1, 2, 4, 8]
}
random_search = RandomizedSearchCV(estimator=rf, param_distributions=param_dist, n_iter=20, cv=5, scoring='f1_macro', n_jobs=-1, verbose=1, random_state=42)
random_search.fit(X_train, y_train)
best_params = random_search.best_params_
best_score = random_search.best_score_

# Cross-validation
cv_scores = cross_val_score(rf, X, y, cv=5, scoring='f1_macro')
mean_cv_score = cv_scores.mean()

```

Feature Selection

python

```

from sklearn.feature_selection import RFE, SelectKBest, chi2, f_classif, mutual_info_classif

# RFE
rfe = RFE(estimator=LogisticRegression(), n_features_to_select=2)
rfe.fit(X, y)
selected_features = rfe.support_
ranking = rfe.ranking_

# SelectKBest
selector = SelectKBest(score_func=chi2, k=2)
X_selected = selector.fit_transform(X, y)
selected_features = selector.get_support()

```

Plotting

python

```

import matplotlib.pyplot as plt
import seaborn as sns

# Histogram
plt.figure(figsize=(10, 6))
sns.histplot(df['col'], bins=30, kde=True)
plt.title('Distribution of Col')
plt.xlabel('Col')
plt.ylabel('Frequency')
plt.show()

```

```
# Box plot
plt.figure(figsize=(10, 6))
sns.boxplot(x=df['col'])
plt.title('Box Plot of Col')
plt.show()

# Scatter plot
plt.figure(figsize=(10, 6))
sns.scatterplot(x=df['col1'], y=df['col2'], hue=df['target'])
plt.title('Scatter Plot')
plt.show()

# Bar plot
plt.figure(figsize=(10, 6))
sns.barplot(x='col', y='target', data=df)
plt.title('Bar Plot')
plt.show()

# Count plot
plt.figure(figsize=(10, 6))
sns.countplot(x='col', data=df)
plt.title('Count Plot')
plt.show()

# Heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.show()

# Pair plot
sns.pairplot(df, hue='target')
plt.show()

# Confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label='ROC Curve (AUC = %0.2f)' % auc)
plt.plot([0, 1], [0, 1], 'k--')
```

```
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()

# Feature importance
plt.figure(figsize=(10, 6))
sns.barplot(x=rf.feature_importances_, y=X.columns)
plt.title('Feature Importance')
plt.show()

# Model comparison
models = ['Logistic Regression', 'Random Forest', 'XGBoost']
scores = [0.85, 0.90, 0.92]
plt.figure(figsize=(10, 6))
sns.barplot(x=models, y=scores)
plt.title('Model Comparison')
plt.xlabel('Model')
plt.ylabel('F1 Score')
plt.show()
```

This comprehensive topic-wise arrangement covers virtually all questions asked across all MLP vivas (Level 1, 2, and 3) from the response sheet. Use it as a study guide, preparing each section thoroughly, and you will be well-equipped for your viva. Good luck!

give answer along with question explained properly , make sure i don't have to look in the chat above to refer the question

MLP Viva – Complete Topic-wise Questions with Detailed Answers

Below is a comprehensive, self-contained guide. Every question is written out fully, followed by a clear, exam-ready answer. You do not need to refer to anything else.

SECTION 1: PROJECT & NOTEBOOK EXPLANATION

Q1. Explain your notebook / project / problem statement.

Answer: My project is based on the [Kaggle competition name] dataset. The problem statement is to predict [target variable] based on the given features. It is a [classification/regression] problem. I started by loading the data, performed EDA to understand distributions and relationships, preprocessed the data (handled missing values, encoded categorical variables, scaled numerical features), engineered new features, trained multiple models, tuned hyperparameters, compared model performance, and selected the best model for final submission.

Q2. What is the objective of this project?

Answer: The objective is to build a machine learning model that can accurately predict [target] for unseen data, minimizing the chosen evaluation metric (e.g., F1 score, RMSLE, RMSE). The goal is to achieve a good leaderboard score while ensuring the model generalizes well and does not overfit.

Q3. What type of ML problem is this – classification or regression? Supervised or unsupervised?

Answer: This is a **supervised learning** problem because we have labeled training data (input features and corresponding target values). Specifically, it is a [classification/regression] problem because the target variable is [categorical/continuous].

- If target is categorical → Classification
- If target is continuous → Regression

Q4. Explain the dataset and its features.

Answer: The dataset contains [N] rows and [M] columns. The features include:

- **Numerical features:** [list them]
- **Categorical features:** [list them]
- **Target variable:** [name], which represents [meaning]

Key observations: [mention missing values, class imbalance, skewness, etc.]

Q5. Show me your EDA / Feature Engineering / Preprocessing / Pipeline / Models / HPT / Model Comparison.

Answer: (You should be ready to screen-share and walk through each section of your notebook. For each section, explain what you did and why.)

- **EDA:** I plotted histograms, box plots, scatter plots, correlation heatmap, and count plots to understand distributions and relationships.
- **Preprocessing:** I handled missing values using [SimpleImputer with median/mode], encoded categorical variables using [OneHotEncoder/LabelEncoder/OrdinalEncoder], and scaled numerical features using [StandardScaler/MinMaxScaler/RobustScaler].
- **Feature Engineering:** I created new features like [list them] because [reason].
- **Models:** I trained [list models] and compared them using [metric].
- **HPT:** I used [GridSearchCV/RandomizedSearchCV] to tune hyperparameters like [list].
- **Model Comparison:** I compared models based on [metric] and selected [best model] because [reason].

Q6. What insights did you get from EDA?

Answer: From EDA, I found:

- The target variable is [normally distributed/right-skewed/left-skewed].
- [Feature X] has a strong positive/negative correlation with the target.
- There are [N] missing values in [columns].
- There is class imbalance (if applicable).
- [Feature Y] has outliers that need treatment.
- [Feature Z] is highly correlated with [Feature W], indicating multicollinearity.

Q7. What difficulties did you face and how did you overcome them?

Answer: I faced:

- Missing values → handled using imputation.
- Class imbalance → handled using `class_weight='balanced'` / SMOTE.
- Overfitting → handled using regularization, cross-validation, and hyperparameter tuning.
- High cardinality categorical features → handled using target encoding / frequency encoding.
- Long training time → used LightGBM instead of XGBoost for speed.

Q8. What would you do differently if you had more time?

Answer: I would:

- Try more feature engineering combinations.
- Use advanced hyperparameter tuning (Bayesian optimization/Optuna).
- Try ensemble stacking/blending.
- Collect more data if possible.
- Deploy the model using FastAPI and Docker.

Q9. How can you further improve your model/score?

Answer: By:

- Adding more relevant features.
- Trying different encoding techniques (target encoding, frequency encoding).
- Using ensemble methods (stacking, blending).
- Tuning hyperparameters more extensively.
- Handling outliers better.
- Trying different models (CatBoost, Neural Networks).

Q10. Did you use any LLM/AI? Where and why?

Answer: (Be honest.) Yes, I used AI for [debugging / understanding concepts / syntax help], but I understood every line of code and can explain it. I did not use it to blindly

generate the entire notebook.

SECTION 2: EXPLORATORY DATA ANALYSIS (EDA)

Q11. What is EDA and why do we do it?

Answer: EDA (Exploratory Data Analysis) is the process of analyzing and visualizing data to understand its main characteristics, distributions, patterns, relationships, and anomalies before modeling. We do it to:

- Understand the data structure.
- Identify missing values, outliers, and imbalances.
- Find relationships between features and target.
- Guide feature engineering and preprocessing decisions.
- Detect data quality issues early.

Q12. What is univariate, bivariate, and multivariate analysis?

Answer:

- **Univariate:** Analysis of a single variable (e.g., histogram of one column).
- **Bivariate:** Analysis of two variables (e.g., scatter plot, correlation between two columns).
- **Multivariate:** Analysis of more than two variables (e.g., pair plot, heatmap of all features).

Q13. What is a box plot? What are its components?

Answer: A box plot (box-and-whisker plot) displays the distribution of numerical data using:

- **Median (Q2):** Middle value.
- **Q1:** 25th percentile.

- **Q3:** 75th percentile.
- **IQR:** $Q3 - Q1$.
- **Whiskers:** Extend to $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$.
- **Outliers:** Points beyond the whiskers.

Q14. What is a histogram? Difference between bar plot and histogram?

Answer:

- **Histogram:** Shows frequency distribution of continuous numerical data; bars touch each other.
- **Bar plot:** Shows frequency/count of categorical data; bars do not touch.

Q15. What is a scatter plot? What does it show?

Answer: A scatter plot shows the relationship between two numerical variables. Each point represents one observation. It helps identify correlation, clusters, and outliers.

Q16. What is a correlation matrix / heatmap?

Answer: A correlation matrix shows the correlation coefficient between all pairs of numerical features. A heatmap visualizes it with colors. Values range from -1 to +1:

- +1: Perfect positive correlation.
- -1: Perfect negative correlation.
- 0: No linear correlation.

Q17. What is multicollinearity? How do you handle it?

Answer: Multicollinearity occurs when two or more features are highly correlated. It can cause instability in linear models. We handle it by:

- Dropping one of the correlated features.
- Using PCA for dimensionality reduction.

- Using regularization (Ridge/Lasso).
- Using tree-based models which are less sensitive.

Q18. What is skewness? How do you handle skewed data?

Answer: Skewness measures asymmetry in data distribution.

- **Right-skewed (positive):** Tail on the right; mean > median.
- **Left-skewed (negative):** Tail on the left; mean < median.

Handling: Log transformation, square root, Box-Cox, or removing outliers.

Q19. What is log transformation? Why use it?

Answer: Log transformation reduces right skewness and compresses large values. It helps linear models by making the distribution more normal and reducing the impact of outliers. $\text{np.log1p}(x) = \log(1+x)$ is used when data contains zeros.

Q20. What are outliers? How do you detect and handle them?

Answer: Outliers are extreme values that deviate significantly from other observations.

- **Detection:** IQR method, Z-score, box plot, scatter plot.
- **Handling:** Remove, cap/floor (winsorization), transform (log), or use robust models (tree-based).

Q21. What is IQR? How is it calculated?

Answer: IQR (Interquartile Range) = $Q3 - Q1$. It represents the middle 50% of data.

- Lower bound = $Q1 - 1.5 \times \text{IQR}$
- Upper bound = $Q3 + 1.5 \times \text{IQR}$
- Values outside these bounds are outliers.

Q22. How do you handle missing values?

Answer: Methods:

- **Remove:** Drop rows/columns if missing is very high.
- **Impute:** Mean/median (numerical), mode (categorical), constant, KNN imputer, iterative imputer.
- **Flag:** Create a binary column indicating missingness.
- **Model-based:** Use algorithms that handle missing values (XGBoost, LightGBM).

Q23. What is SimpleImputer? What are its strategies?

Answer: `SimpleImputer` from `sklearn` replaces missing values.

- `strategy='mean'` : Replace with mean (numerical).
- `strategy='median'` : Replace with median (numerical, robust to outliers).
- `strategy='most_frequent'` : Replace with mode (categorical).
- `strategy='constant'` : Replace with a constant value.

Q24. When do you use mean, median, and mode for imputation?

Answer:

- **Mean:** When data is normally distributed and no outliers.
- **Median:** When data is skewed or has outliers.
- **Mode:** For categorical variables.

Q25. What is class imbalance? How do you handle it?

Answer: Class imbalance occurs when one class has significantly more samples than another.

Handling:

- **Resampling:** Oversampling (SMOTE), undersampling.
- **Class weights:** `class_weight='balanced'` .

- **Metrics:** Use F1, precision, recall, ROC-AUC instead of accuracy.
- **Algorithms:** Use tree-based models that handle imbalance.

Q26. What is SMOTE?

Answer: SMOTE (Synthetic Minority Oversampling Technique) creates synthetic samples for the minority class by interpolating between existing minority samples. It helps balance the dataset without duplicating data.

Q27. What is data leakage? How do you prevent it?

Answer: Data leakage occurs when information from the test set leaks into the training process, giving overly optimistic results.

Prevention:

- Split data before preprocessing.
- Fit transformers only on training data.
- Use pipelines.
- Avoid using target-derived features in preprocessing.

Q28. Should train-test split be done before or after preprocessing?

Answer: Before preprocessing. If you preprocess before splitting, statistics like mean/median from the test set leak into training. Always split first, then fit preprocessing on training only and transform test.

SECTION 3: DATA PREPROCESSING

Q29. What is data preprocessing? Why is it required?

Answer: Data preprocessing is the process of cleaning and transforming raw data into a format suitable for machine learning. It is required because:

- Models cannot handle missing values, categorical text, or unscaled features well.
- It improves model accuracy and convergence.
- It reduces noise and redundancy.

Q30. What are the steps in data preprocessing?

Answer:

1. Handle missing values (imputation/removal).
2. Handle outliers (detection/treatment).
3. Encode categorical variables (OHE, Label, Ordinal).
4. Scale numerical features (StandardScaler, MinMaxScaler).
5. Feature engineering (create new features).
6. Feature selection (RFE, SelectKBest).
7. Split data (train-test/validation).

Q31. What is feature scaling? Why is it important?

Answer: Feature scaling transforms numerical features to a similar range. It is important because:

- Distance-based models (KNN, SVM) are sensitive to scale.
- Gradient descent converges faster.
- Regularization works properly.
- Prevents features with larger values from dominating.

Q32. What is StandardScaler? How does it work?

Answer: StandardScaler standardizes features by removing the mean and scaling to unit variance:

$$z = (x - \mu) / \sigma$$

- Mean becomes 0.
- Standard deviation becomes 1.

- Range: approximately -3 to +3 for normal data.

Q33. What is MinMaxScaler? How does it work?

Answer: MinMaxScaler scales features to a fixed range [0, 1]:

$$x_scaled = (x - \min) / (\max - \min)$$

- Sensitive to outliers because it uses min and max.

Q34. What is RobustScaler? How does it work?

Answer: RobustScaler scales using median and IQR:

$$x_scaled = (x - \text{median}) / \text{IQR}$$

- Robust to outliers because it does not use min/max.

Q35. Difference between StandardScaler and MinMaxScaler.

Answer:

- StandardScaler: mean=0, std=1, range varies, less affected by outliers.
- MinMaxScaler: range [0,1], affected by outliers.

Q36. Difference between StandardScaler and RobustScaler.

Answer:

- StandardScaler uses mean and std → affected by outliers.
- RobustScaler uses median and IQR → robust to outliers.

Q37. Which models require scaling? Which do not?

Answer:

- **Require scaling:** KNN, SVM, K-Means, PCA, Logistic Regression, Linear Regression, Neural Networks.
- **Do not require scaling:** Decision Trees, Random Forest, XGBoost, LightGBM, CatBoost.

Q38. What is encoding? Why do we need it?

Answer: Encoding converts categorical variables into numerical format because ML models cannot process text directly.

Q39. What is One-Hot Encoding (OHE)?

Answer: OHE creates a binary column for each category. For a feature with categories [A, B, C], it creates 3 columns. It is used for nominal categorical data with low cardinality.

Q40. What is Label Encoding?

Answer: Label Encoding assigns a unique integer to each category (A=0, B=1, C=2). It is used for ordinal data or tree-based models. It can mislead linear models into thinking there is an order.

Q41. What is Ordinal Encoding?

Answer: Ordinal Encoding assigns integers based on a defined order (e.g., Low=0, Medium=1, High=2). Used when categories have a natural order.

Q42. What is Target Encoding?

Answer: Target Encoding replaces categories with the mean of the target variable for that category. It is useful for high cardinality features but can cause data leakage if not done with cross-validation.

Q43. Difference between One-Hot Encoding and Label Encoding.

Answer:

- OHE: Creates binary columns, no order implied, increases dimensionality.
- Label: Assigns integers, implies order, keeps dimensionality low.

Q44. What is high cardinality? How does it affect OHE?

Answer: High cardinality means a categorical feature has many unique values. OHE creates too many columns, leading to the curse of dimensionality. Use target encoding or frequency encoding instead.

Q45. What is `handle_unknown='ignore'` in `OneHotEncoder`?

Answer: It tells the encoder to ignore unknown categories during transform (instead of throwing an error) and encode them as all zeros.

Q46. What is `sparse_output=False` in `OneHotEncoder`?

Answer: It returns a dense NumPy array instead of a sparse matrix. Useful when you want to convert to a DataFrame or when memory is not an issue.

Q47. What is a Pipeline? Why do we use it?

Answer: A Pipeline chains multiple preprocessing steps and a model into one object.
Benefits:

- Prevents data leakage.
- Simplifies code.
- Makes cross-validation and hyperparameter tuning easier.
- Ensures same transformations are applied to train and test.

Q48. What is ColumnTransformer?

Answer: ColumnTransformer applies different transformers to different columns. For example, scaling numerical columns and encoding categorical columns simultaneously.

Q49. Difference between Pipeline and ColumnTransformer.

Answer:

- Pipeline: Sequentially applies transformers to the entire dataset.
- ColumnTransformer: Applies different transformers to different column subsets in parallel.

Q50. What is `fit_transform()` vs `transform()` ?

Answer:

- `fit_transform()` : Learns parameters (mean, std, categories) from data and applies transformation. Used on training data.
- `transform()` : Applies learned parameters without re-learning. Used on test/validation data.

Q51. What is feature engineering?

Answer: Feature engineering is creating new features from existing ones to improve model performance. Examples: extracting year from date, creating interaction features, binning, log transformation.

Q52. What is feature selection?

Answer: Feature selection is selecting the most relevant features and dropping irrelevant/redundant ones. Methods:

- Filter: Correlation, chi-square, ANOVA.
- Wrapper: RFE, forward/backward selection.

- Embedded: Lasso, tree-based feature importance.

Q53. What is RFE?

Answer: Recursive Feature Elimination (RFE) recursively removes the least important features based on model coefficients/importance until the desired number of features remains.

Q54. What is PCA? How does it work?

Answer: PCA (Principal Component Analysis) is a dimensionality reduction technique. It:

1. Standardizes the data.
2. Computes the covariance matrix.
3. Finds eigenvalues and eigenvectors.
4. Sorts eigenvectors by eigenvalue (largest = most variance).
5. Projects data onto top k eigenvectors.

It reduces dimensions while preserving maximum variance. It is unsupervised and linear.

Q55. Does PCA discard data points?

Answer: No, PCA does not discard data points. It transforms the data into a new coordinate system with fewer dimensions. Some variance is lost, but no rows are removed.

Q56. Can you get back the original data after PCA?

Answer: You can approximately reconstruct the original data using inverse transform, but you cannot get back the exact original data because some information (variance) is lost.

Q57. What is the curse of dimensionality?

Answer: As the number of features increases, the data becomes sparse in high-dimensional space. Models need exponentially more data to generalize well. Distance-based algorithms suffer the most.

SECTION 4: MACHINE LEARNING MODELS

Q58. What is the difference between parameters and hyperparameters?

Answer:

- **Parameters:** Learned by the model during training (e.g., weights, coefficients).
- **Hyperparameters:** Set before training (e.g., learning rate, n_estimators, max_depth).

Q59. What is the difference between parametric and non-parametric models?

Answer:

- **Parametric:** Assumes a fixed functional form; fixed number of parameters (e.g., Linear Regression, Logistic Regression).
- **Non-parametric:** No fixed form; number of parameters grows with data (e.g., KNN, Decision Trees, Random Forest).

Q60. What is a baseline model?

Answer: A simple model used as a reference point. Examples: predicting the majority class, mean value, or using Logistic Regression/Linear Regression without tuning.

Q61. What is bagging?

Answer: Bagging (Bootstrap Aggregating) trains multiple models on different bootstrap samples of the data and averages their predictions. It reduces variance and prevents overfitting. Example: Random Forest.

Q62. What is boosting?

Answer: Boosting trains models sequentially, where each new model focuses on the errors of the previous ones. It reduces bias and can achieve high accuracy. Examples: AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost.

Q63. Difference between bagging and boosting.

Answer:

- Bagging: Parallel training, reduces variance, equal weights.
- Boosting: Sequential training, reduces bias, weighted based on performance.

Q64. Which reduces bias and which reduces variance?

Answer:

- Bagging reduces variance.
- Boosting reduces bias (and also variance to some extent).

Q65. What is stacking?

Answer: Stacking combines multiple models (base learners) by training a meta-model on their predictions. The meta-model learns how to best combine the base models' outputs.

Q66. What is blending?

Answer: Blending is similar to stacking but uses a hold-out validation set instead of cross-validation to train the meta-model.

Q67. What is Linear Regression?

Answer: Linear Regression models the relationship between input features and a continuous target by fitting a linear equation:

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

It minimizes the Mean Squared Error (MSE).

Q68. What is Logistic Regression?

Answer: Logistic Regression is a classification algorithm that models the probability of a binary outcome using the sigmoid function:

$$P(y=1) = 1 / (1 + e^{(-z)}), \text{ where } z = w_0 + w_1x_1 + \dots$$

It outputs probabilities between 0 and 1 and uses log loss (binary cross-entropy) as the loss function.

Q69. Why is Logistic Regression called "regression" when it is used for classification?

Answer: It is called regression because it models the log-odds of the probability as a linear combination of features:

$$\log(p/(1-p)) = w_0 + w_1x_1 + \dots$$

The output is a continuous probability, which is then thresholded for classification.

Q70. What is the loss function of Logistic Regression?

Answer: Log Loss (Binary Cross-Entropy):

$$L = -[y \log(p) + (1-y) \log(1-p)]$$

It penalizes wrong predictions heavily.

Q71. What is the sigmoid function?

Answer: $\sigma(z) = 1 / (1 + e^{(-z)})$. It maps any real number to (0, 1). Used in Logistic Regression to convert linear output to probability.

Q72. What is Gradient Descent?

Answer: Gradient Descent is an optimization algorithm that iteratively updates parameters in the direction of the negative gradient of the loss function to minimize it:

$$w = w - \eta \times \partial L / \partial w$$

where η is the learning rate.

Q73. What is the learning rate? What happens if it's too high or too low?

Answer: Learning rate (η) controls the step size in gradient descent.

- Too high: Overshoots minimum, may diverge.
- Too low: Slow convergence, may get stuck in local minima.

Q74. What is regularization? L1 vs L2?

Answer: Regularization adds a penalty to the loss function to prevent overfitting.

- **L1 (Lasso):** Adds $\lambda \sum |w|$. Can shrink coefficients to zero → feature selection.
- **L2 (Ridge):** Adds $\lambda \sum w^2$. Shrinks coefficients but not to zero → prevents overfitting.

Q75. What is ElasticNet?

Answer: ElasticNet combines L1 and L2 penalties:

$$\text{Loss} + \lambda_1 \sum |w| + \lambda_2 \sum w^2$$

It is useful when there are multiple correlated features.

Q76. What is a Decision Tree?

Answer: A Decision Tree splits data recursively based on feature thresholds to minimize impurity (Gini/Entropy). Each internal node represents a decision, each leaf represents a prediction.

Q77. How is the root node selected in a Decision Tree?

Answer: The root node is selected as the feature and threshold that gives the highest information gain (or lowest Gini impurity) for the split.

Q78. What is entropy?

Answer: Entropy measures impurity/uncertainty:

$$\text{Entropy} = -\sum p_i \log_2(p_i)$$

- 0 for pure node.
- 1 for 50-50 binary split.

Q79. What is Gini index?

Answer: $\text{Gini} = 1 - \sum p_i^2$. It measures impurity. Range: 0 (pure) to 0.5 (binary 50-50). Lower is better.

Q80. What is information gain?

Answer: $\text{Information Gain} = \text{Entropy}(\text{parent}) - \sum (\text{weighted Entropy}(\text{children}))$. It measures the reduction in entropy after a split. Higher is better.

Q81. What is Random Forest?

Answer: Random Forest is an ensemble of Decision Trees trained on bootstrap samples with random feature subsets. It averages predictions (regression) or takes majority vote (classification). It reduces variance and overfitting.

Q82. Is Random Forest bagging or boosting?

Answer: Bagging. It trains trees in parallel on bootstrap samples.

Q83. What is `n_estimators` in Random Forest?

Answer: Number of trees in the forest. More trees → better performance but slower. Typically 100-500.

Q84. What is `max_depth` in Random Forest?

Answer: Maximum depth of each tree. Controls complexity. Deeper trees → more overfitting.

Q85. What is OOB score?

Answer: Out-of-Bag score is the accuracy of each tree on the samples not included in its bootstrap sample. It provides a free validation estimate.

Q86. What is Gradient Boosting?

Answer: Gradient Boosting builds trees sequentially, where each tree fits the residual errors of the previous ensemble. It minimizes a loss function using gradient descent.

Q87. What is XGBoost?

Answer: XGBoost (Extreme Gradient Boosting) is an optimized implementation of Gradient Boosting with:

- Regularization (L1/L2).
- Parallel processing.
- Handling missing values.
- Tree pruning.
- Second-order Taylor expansion for faster convergence.

Q88. What is LightGBM?

Answer: LightGBM is a fast Gradient Boosting framework by Microsoft. It uses:

- Leaf-wise tree growth (instead of level-wise).
- GOSS (Gradient-based One-Side Sampling).
- EFB (Exclusive Feature Bundling).

It is faster and more memory-efficient than XGBoost.

Q89. What is CatBoost?

Answer: CatBoost is a Gradient Boosting framework by Yandex. It handles categorical features natively using ordered target encoding and uses ordered boosting to reduce overfitting.

Q90. Difference between XGBoost and LightGBM.

Answer:

- XGBoost: Level-wise tree growth, slower, more accurate on small data.
- LightGBM: Leaf-wise tree growth, faster, better for large data.

Q91. Difference between bagging and boosting.

Answer:

- Bagging: Parallel, reduces variance, equal weights.
- Boosting: Sequential, reduces bias, weighted by performance.

Q92. What is KNN?

Answer: K-Nearest Neighbors classifies a point based on the majority class of its K nearest neighbors. For regression, it averages the K nearest values.

Q93. Why is KNN called a lazy learner?

Answer: Because it does not learn a model during training. It stores all training data and performs computation only at prediction time.

Q94. What is SVM?

Answer: Support Vector Machine finds the hyperplane that best separates classes with maximum margin. It uses the kernel trick to handle non-linear data.

Q95. What is the kernel trick?

Answer: The kernel trick implicitly maps data to a higher-dimensional space where it becomes linearly separable, without explicitly computing the transformation. Common kernels: linear, polynomial, RBF, sigmoid.

Q96. What is the C parameter in SVM?

Answer: C controls the trade-off between maximizing the margin and minimizing classification error.

- High C: Low margin, fewer misclassifications, may overfit.
- Low C: Large margin, more misclassifications, may underfit.

Q97. What is the gamma parameter in SVM?

Answer: Gamma defines the influence of a single training point.

- High gamma: Close influence, may overfit.
- Low gamma: Far influence, may underfit.

Q98. What is Naive Bayes?

Answer: Naive Bayes is a probabilistic classifier based on Bayes' theorem, assuming features are independent. It is fast and works well for text classification.

Q99. Why is it called "Naive"?

Answer: Because it assumes all features are independent of each other, which is rarely true in real data.

Q100. What is K-Means Clustering?

Answer: K-Means partitions data into K clusters by:

1. Randomly initializing K centroids.
2. Assigning each point to the nearest centroid.
3. Updating centroids to the mean of assigned points.
4. Repeating until convergence.

Q101. What is the Elbow method?

Answer: The Elbow method plots the within-cluster sum of squares (WCSS) against K. The "elbow" point (where the curve bends) suggests the optimal K.

Q102. Difference between K-Means and KNN.

Answer:

- K-Means: Unsupervised clustering, finds K clusters.
- KNN: Supervised classification/regression, uses K nearest neighbors.

Q103. What is the difference between supervised and unsupervised learning?

Answer:

- Supervised: Labeled data, predicts target (classification/regression).
- Unsupervised: Unlabeled data, finds patterns (clustering, dimensionality reduction).

Q104. What is reinforcement learning?

Answer: An agent learns by interacting with an environment, receiving rewards/penalties, and optimizing its policy to maximize cumulative reward.

Q105. What is a neural network?

Answer: A neural network is a series of layers of interconnected nodes (neurons). Each neuron computes a weighted sum of inputs, adds a bias, and applies an activation function. Deep networks learn hierarchical representations.

Q106. What are activation functions?

Answer: Activation functions introduce non-linearity:

- **ReLU:** $\max(0, z)$ – fast, prevents vanishing gradient.
- **Sigmoid:** $1/(1+e^{(-z)})$ – output (0,1), used in binary classification.
- **Tanh:** $(e^z - e^{(-z)})/(e^z + e^{(-z)})$ – output (-1,1).
- **Softmax:** Used for multi-class classification.

SECTION 5: MODEL EVALUATION

Q107. What is accuracy? Why is it not good for imbalanced data?

Answer: Accuracy = $(TP + TN) / \text{Total}$. It is misleading for imbalanced data because a model predicting only the majority class can have high accuracy but fail on the minority class.

Q108. What is precision?

Answer: Precision = $TP / (TP + FP)$. It measures how many predicted positives are actually positive. High precision = few false positives.

Q109. What is recall?

Answer: Recall = $TP / (TP + FN)$. It measures how many actual positives are correctly identified. High recall = few false negatives.

Q110. What is F1 score?

Answer: $F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$. It is the harmonic mean of precision and recall. Good for imbalanced data.

Q111. When do you use precision over recall?

Answer: When false positives are more costly (e.g., spam detection – you don't want to mark important emails as spam).

Q112. When do you use recall over precision?

Answer: When false negatives are more costly (e.g., cancer detection – you don't want to miss a cancer patient).

Q113. What is a confusion matrix?

Answer: A table showing TP, TN, FP, FN:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Q114. What is ROC-AUC?

Answer: ROC curve plots TPR vs FPR at different thresholds. AUC (Area Under Curve) measures the model's ability to distinguish between classes. Range: 0.5 (random) to 1

(perfect).

Q115. What is R^2 score?

Answer: $R^2 = 1 - (SS_{\text{res}} / SS_{\text{tot}})$. It measures how well the model explains variance in the target. Range: $-\infty$ to 1. 1 = perfect, 0 = same as mean, negative = worse than mean.

Q116. What is RMSE?

Answer: $RMSE = \sqrt{MSE} = \sqrt{(\sum (y - \hat{y})^2 / n)}$. It measures average prediction error in the same units as the target.

Q117. What is RMSLE?

Answer: $RMSLE = \sqrt{(\sum (\log(y+1) - \log(\hat{y}+1))^2 / n)}$. It is used when the target is skewed or when relative error matters more than absolute error.

Q118. Difference between RMSE and RMSLE.

Answer:

- RMSE: Penalizes large errors heavily, sensitive to outliers.
- RMSLE: Penalizes relative errors, robust to outliers, used for skewed targets.

Q119. What is cross-validation?

Answer: Cross-validation splits data into K folds. The model trains on K-1 folds and validates on the remaining fold, repeating K times. It gives a more reliable estimate of model performance.

Q120. What is K-Fold cross-validation?

Answer: Data is divided into K equal folds. For each fold, train on K-1 folds and validate on the remaining one. Average the K scores. Common K: 5 or 10.

Q121. What is Stratified K-Fold?

Answer: Same as K-Fold but preserves the class proportion in each fold. Used for imbalanced classification.

Q122. What is overfitting?

Answer: Model performs well on training data but poorly on unseen data. It memorizes noise. Signs: high train score, low validation score.

Q123. What is underfitting?

Answer: Model is too simple to capture patterns. Performs poorly on both training and validation data.

Q124. How do you detect overfitting/underfitting?

Answer: Compare training and validation scores:

- Overfitting: Train $\gg$ Validation.
- Underfitting: Train $\approx$ Validation, both low.
- Good fit: Train $\approx$ Validation, both high.

Q125. How do you prevent overfitting?

Answer:

- More data.
- Regularization (L1/L2).
- Cross-validation.

- Early stopping.
- Pruning (trees).
- Dropout (neural networks).
- Ensemble methods.
- Reduce model complexity.

Q126. What is the bias-variance tradeoff?

Answer:

- **Bias:** Error from overly simple assumptions (underfitting).
- **Variance:** Error from sensitivity to training data (overfitting).
- Total error = $\text{Bias}^2 + \text{Variance} + \text{Irreducible error}$.
- Need to balance both for optimal performance.

Q127. What is GridSearchCV?

Answer: GridSearchCV exhaustively tries all combinations of hyperparameters in a grid and evaluates each using cross-validation. Returns the best combination. Slow but thorough.

Q128. What is RandomizedSearchCV?

Answer: RandomizedSearchCV randomly samples hyperparameter combinations from a distribution for a fixed number of iterations (`n_iter`). Faster than GridSearch, may find good enough parameters.

Q129. Difference between GridSearchCV and RandomizedSearchCV.

Answer:

- GridSearch: Tries all combinations → exhaustive, slow.
- RandomizedSearch: Tries random combinations → faster, may miss optimal.

Q130. How many models will be trained in GridSearchCV?

Answer: Number of models = (product of all hyperparameter values) × cv. For example, $3 \times 3 \times 2 = 18$ combinations × 5 cv = 90 models.

Q131. How many models will be trained in RandomizedSearchCV?

Answer: Number of models = `n_iter` × cv. For example, `n_iter=20`, `cv=5` → 100 models.

Q132. What is `n_iter` in RandomizedSearchCV?

Answer: Number of random hyperparameter combinations to try.

Q133. What is `cv` in GridSearchCV?

Answer: Number of cross-validation folds. Default is 5.

Q134. What is `scoring` in GridSearchCV?

Answer: The metric used to evaluate models. Examples: 'accuracy', 'f1_macro', 'roc_auc', 'neg_mean_squared_error'.

Q135. What is `n_jobs` in GridSearchCV?

Answer: Number of parallel jobs. `n_jobs=-1` uses all available cores.

Q136. What is `verbose` in GridSearchCV?

Answer: Controls the amount of output printed during fitting. Higher = more detailed logs.

SECTION 6: STATISTICS & MATHEMATICS

Q137. What is mean, median, and mode?

Answer:

- **Mean:** Average = $\Sigma x / n$. Sensitive to outliers.
- **Median:** Middle value when sorted. Robust to outliers.
- **Mode:** Most frequent value. Used for categorical data.

Q138. What is variance and standard deviation?

Answer:

- **Variance:** Average squared deviation from mean = $\Sigma(x - \mu)^2 / n$.
- **Standard Deviation:** Square root of variance = $\sqrt{\text{Variance}}$. Same units as data.

Q139. What is a normal distribution?

Answer: A symmetric bell-shaped distribution where mean = median = mode. 68% of data within 1σ , 95% within 2σ , 99.7% within 3σ (empirical rule).

Q140. What is the Central Limit Theorem?

Answer: The sampling distribution of the sample mean approaches a normal distribution as sample size increases, regardless of the population distribution.

Q141. What is a p-value?

Answer: The probability of observing results at least as extreme as the observed, assuming the null hypothesis is true. If $p < \alpha$ (usually 0.05), reject the null hypothesis.

Q142. What is a t-test?

Answer: A statistical test comparing means of two groups. Types: one-sample, independent two-sample, paired.

Q143. What is a chi-square test?

Answer: A test for independence between two categorical variables. Compares observed vs expected frequencies.

Q144. What is ANOVA?

Answer: Analysis of Variance tests whether means of three or more groups are significantly different.

Q145. What is correlation?

Answer: Correlation measures the strength and direction of a linear relationship between two variables. Range: -1 to +1.

Q146. What is the Pearson correlation coefficient?

Answer: $r = \text{Cov}(x,y) / (\sigma_x \times \sigma_y)$. Measures linear correlation. Assumes normality and linearity.

Q147. What is the difference between correlation and causation?

Answer: Correlation means two variables move together. Causation means one causes the other. Correlation does not imply causation.

Q148. What is Bayes' theorem?

Answer: $P(A|B) = [P(B|A) \times P(A)] / P(B)$. It updates the probability of a hypothesis based on new evidence.

Q149. What is a Type I error?

Answer: Rejecting a true null hypothesis (false positive).

Q150. What is a Type II error?

Answer: Failing to reject a false null hypothesis (false negative).

SECTION 7: PYTHON & CODING

Q151. How do you load a dataset in pandas?

Answer:

```
python

import pandas as pd
df = pd.read_csv('train.csv')
```

Q152. How do you display the first 5 rows?

Answer:

```
python

df.head()
```

Q153. How do you check for null values?

Answer:

python

```
df.isnull().sum()
```

Q154. How do you drop a column?

Answer:

python

```
df.drop(columns=['col_name'], inplace=True)
```

Q155. How do you filter rows based on a condition?

Answer:

python

```
df_filtered = df[df['col'] > 50]
```

Q156. How do you select only numerical columns?

Answer:

python

```
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
```

Q157. How do you select only categorical columns?

Answer:

python

```
cat_cols = df.select_dtypes(include=['object']).columns
```

Q158. How do you find unique values in a column?

Answer:

python

```
df['col'].unique()
```

Q159. How do you count the frequency of each unique value?

Answer:

python

```
df['col'].value_counts()
```

Q160. How do you replace '?' with NaN?

Answer:

python

```
df.replace('?', np.nan, inplace=True)
```

Q161. How do you create a train-test split?

Answer:

python

```
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Q162. How do you create a pipeline?

Answer:

```
python

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])
pipeline.fit(X_train, y_train)
```

Q163. How do you create a ColumnTransformer?**Answer:**

```
python

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(), cat_cols)
])
```

Q164. How do you create a SimpleImputer?**Answer:**

```
python

from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='median')
X_imputed = imputer.fit_transform(X)
```

Q165. How do you create a StandardScaler?

Answer:

```
python

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Q166. How do you create a OneHotEncoder?**Answer:**

```
python

from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
X_encoded = ohe.fit_transform(X_cat)
```

Q167. How do you create a LinearRegression model?**Answer:**

```
python

from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)
```

Q168. How do you create a LogisticRegression model?**Answer:**

```
python

from sklearn.linear_model import LogisticRegression
log_reg = LogisticRegression(max_iter=1000, random_state=42)
log_reg.fit(X_train, y_train)
y_pred = log_reg.predict(X_test)
```

Q169. How do you create a RandomForestClassifier?**Answer:**

python

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
```

Q170. How do you create a GridSearchCV?**Answer:**

python

```
from sklearn.model_selection import GridSearchCV
param_grid = {'n_estimators': [100, 200], 'max_depth': [5, 10]}
grid = GridSearchCV(RandomForestClassifier(), param_grid, cv=5, scoring='f1_macro')
grid.fit(X_train, y_train)
print(grid.best_params_)
```

Q171. How do you create a RandomizedSearchCV?**Answer:**

python

```
from sklearn.model_selection import RandomizedSearchCV
param_dist = {'n_estimators': [100, 200, 300], 'max_depth': [5, 10, None]}
random = RandomizedSearchCV(RandomForestClassifier(), param_dist, n_iter=10, cv=5, scoring='f1_macro', random_state=42)
random.fit(X_train, y_train)
print(random.best_params_)
```

Q172. How do you calculate accuracy?

Answer:

python

```
from sklearn.metrics import accuracy_score  
acc = accuracy_score(y_test, y_pred)
```

Q173. How do you create a confusion matrix?**Answer:**

python

```
from sklearn.metrics import confusion_matrix  
cm = confusion_matrix(y_test, y_pred)
```

Q174. How do you create a classification report?**Answer:**

python

```
from sklearn.metrics import classification_report  
print(classification_report(y_test, y_pred))
```

Q175. How do you plot a histogram?**Answer:**

python

```
import matplotlib.pyplot as plt  
import seaborn as sns  
sns.histplot(df['col'], bins=30, kde=True)  
plt.show()
```

Q176. How do you plot a correlation heatmap?

Answer:

python

```
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')  
plt.show()
```

Q177. How do you plot a box plot?**Answer:**

python

```
sns.boxplot(x=df['col'])  
plt.show()
```

Q178. How do you implement train_test_split from scratch?**Answer:**

python

```
import numpy as np  
  
def train_test_split_scratch(X, y, test_size=0.2, random_state=42):  
    np.random.seed(random_state)  
    indices = np.random.permutation(len(X))  
    test_count = int(len(X) * test_size)  
    test_idx = indices[:test_count]  
    train_idx = indices[test_count:]  
    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]
```

Q179. How do you implement SimpleImputer from scratch?**Answer:**

python

```
def simple_imputer_scratch(X, strategy='mean'):  
    if strategy == 'mean':
```

```

        fill_value = np.nanmean(X, axis=0)
    elif strategy == 'median':
        fill_value = np.nanmedian(X, axis=0)
    elif strategy == 'most_frequent':
        fill_value = np.array([np.bincount(X[:, i][~np.isnan(X[:, i])].astype(int)).argmax() for i in range(X.shape[1])])
    X_imputed = X.copy()
    for i in range(X.shape[1]):
        X_imputed[np.isnan(X[:, i]), i] = fill_value[i]
    return X_imputed

```

Q180. How do you implement StandardScaler from scratch?

Answer:

python

```

def standard_scaler_scratch(X):
    mean = np.mean(X, axis=0)
    std = np.std(X, axis=0)
    return (X - mean) / std

```

Q181. How do you implement Logistic Regression from scratch?

Answer:

python

```

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def logistic_regression_scratch(X, y, lr=0.01, epochs=1000):
    m, n = X.shape
    weights = np.zeros(n)
    bias = 0
    for _ in range(epochs):
        z = np.dot(X, weights) + bias
        y_pred = sigmoid(z)
        dw = (1/m) * np.dot(X.T, (y_pred - y))
        db = (1/m) * np.sum(y_pred - y)
        weights -= lr * dw

```

```
    bias -= lr * db
    return weights, bias
```

Q182. Write a Python program for GCD.

Answer:

python

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
```

Q183. Write a Python program for Fibonacci series.

Answer:

python

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        print(a, end=' ')
        a, b = b, a + b
```

Q184. Write a Python program to check if a number is prime.

Answer:

python

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

SECTION 8: DEPLOYMENT & PRODUCTION

Q185. How would you deploy this model?

Answer: I would:

1. Save the model using `joblib` or `pickle`.
2. Create a FastAPI/Flask app with an endpoint for predictions.
3. Containerize with Docker.
4. Deploy on cloud (AWS/GCP/Azure) or use Kubernetes for scaling.
5. Set up monitoring for model drift and data drift.

Q186. What is FastAPI?

Answer: FastAPI is a modern, fast Python web framework for building APIs. It supports async, automatic documentation, and is ideal for ML model serving.

Q187. What is Docker?

Answer: Docker is a containerization platform that packages an application and its dependencies into a portable container. It ensures consistency across environments.

Q188. What is Kubernetes?

Answer: Kubernetes is an orchestration platform for managing containerized applications. It handles scaling, load balancing, and self-healing.

Q189. What is model drift?

Answer: Model drift occurs when the model's performance degrades over time due to changes in data distribution (data drift) or the relationship between features and target (concept drift).

Q190. How do you handle 500GB of data on a laptop?

Answer: Use chunking (`pandas chunksize`), parallel processing (Dask, Vaex), or use cloud services. For ML, use incremental learning (SGDClassifier with `partial_fit`) or subsample the data.

SECTION 9: GENERAL & HR QUESTIONS

Q191. Introduce yourself.

Answer: (Prepare a 1-2 minute intro covering your name, educational background, current role/studies, interest in ML, and why you took this course.)

Q192. What are your future plans?

Answer: (Be honest – e.g., pursue a master's in ML/AI, work as a data scientist, etc. Mention how this course helps you achieve your goals.)

Q193. What did you learn from this project?

Answer: I learned the complete ML pipeline – from EDA and preprocessing to model building, hyperparameter tuning, and evaluation. I also learned the importance of data leakage prevention, handling imbalanced data, and selecting the right metric.

Q194. How would you explain this project to a non-technical person?

Answer: I would say: "I built a system that looks at historical data and learns patterns to predict [target]. For example, it's like teaching a computer to recognize what makes a customer likely to buy a product, based on past behavior."

SECTION 10: FORMULAS TO REMEMBER

Metric	Formula
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$
Precision	$TP / (TP + FP)$
Recall	$TP / (TP + FN)$
F1 Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$
MSE	$(1/n) \times \sum (y - \hat{y})^2$
RMSE	$\sqrt{\text{MSE}}$
MAE	$(1/n) \times \sum y - \hat{y} $
R ²	$1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$
RMSLE	$\sqrt{[(1/n) \times \sum (\log(y+1) - \log(\hat{y}+1))^2]}$
Pearson Correlation	$\text{Cov}(x, y) / (\sigma_x \times \sigma_y)$
Z-score	$(x - \mu) / \sigma$
IQR	$Q3 - Q1$
Sigmoid	$1 / (1 + e^{(-z)})$
Tanh	$(e^z - e^{(-z)}) / (e^z + e^{(-z)})$
ReLU	$\max(0, z)$
Entropy	$-\sum p_i \log_2(p_i)$
Gini	$1 - \sum p_i^2$
Information Gain	$\text{Entropy}(\text{parent}) - \sum (\text{weighted Entropy}(\text{children}))$
L1 (Lasso)	$\text{Loss} + \lambda \sum w $
L2 (Ridge)	$\text{Loss} + \lambda \sum w^2$

Metric	Formula
Gradient Descent Update	$w = w - \eta \times \partial L / \partial w$

SECTION 11: PROCTOR-SPECIFIC PATTERNS (QUICK REFERENCE)

Proctor ID	Style	Key Focus	Coding?
level1_118	Chill	Notebook scroll, EDA graphs, parametric models	Yes, basic
mlp_level1_viva7	Very chill	Notebook explanation, GridSearch vs Random	Yes, basic
mlp_level1_viva9	Sweet	Notebook, R ² , bagging/boosting, bias-variance	Yes, pipeline
mlp_level1_viva13	Rubric checker	Box plot, n_jobs, reg_alpha, subsample	Yes, train_test_spli
mlp_level1_viva14	Very chill	Notebook, CV, learning rate, overfitting	Yes, basic
level2_74	Repetitive	Preprocessing, scaling, HPT, random_state=42	No
level2_81	Thorough	EDA, encoding, scaling, TF-IDF, models, RFE	Yes, RFE
level2_106	Chill	Notebook, loss function, HPT, F1	No
level2_107	Deep theory	Imputation, outliers, models, MNIST, NN	Yes, MNIST
level2_123	Chill	Notebook, learning rate, n-grams, libraries	Yes, basic

Proctor ID	Style	Key Focus	Coding?
level2_124	Chill	RFE, filtering, correlation, HPT	Yes, dataset link
level3_9	Theory heavy	Preprocessing, PCA, loss, overfitting	No
level3_20	Very chill	Notebook, class imbalance, HPT, leaderboard	No
level3_29	Deep theory + coding	ML types, K-Means, Decision Tree, SVM, PCA	Yes, Iris, RF, PCA
level3_31	Sweet	Notebook, mean/median/mode, HPT, career	No
level3_44	Supportive	Notebook, feature engineering, pipeline, R^2	Yes, imputation
proctor776	Calm	Bias-variance, GOSS, XGBoost, deployment	Yes, train_test_spli
proctor779	Encouraging	Notebook, EDA, feature engineering, parameters	Yes, mode/frequency
proctor782	Strict	Everything, EDA, models, HPT, live coding	Yes, histogram
proctor784	Very strict	Metrics, gradient descent, bagging/boosting	No
proctor786	Thorough	Box plot, correlation, target encoding, formulas	Yes, GCD
proctor787	Chill	EDA, charts, models, gradient boosting	No
proctor788	Friendly	Encoders, target encoder, Ridge, Linear	No
proctor790	Chill	PCA, confusion matrix, log transformation	No

Proctor ID	Style	Key Focus	Coding?
proctor601	Patient	Missing values, encoding, HPT, model comparison	Yes, basic
proctor610	Professional	Classification vs regression, scaling, leakage	Yes, Iris
proctor746	Chill	Notebook, skewness, correlation, K-Fold	No
proctor727	Deep	Manual DataFrame coding, models, hyperparameters	Yes, from scratch
proctor785	Sadistic	Tries to confuse, expects data scientist level	No
proctor774	Very chill	Bagging/boosting, Logistic Regression, RMSLE	No
level3_28	Sweet	EDA, visuals, graphs, model comparison	No
level3_30	Theory heavy	Imbalance, correlation, PCA, L1/L2	No
level3_32	Theory + visuals	Descriptive stats, Logistic Regression, TF-IDF	Yes, plots

SECTION 12: FINAL TIPS

1. **Know your notebook inside out** – Every line, every parameter, every decision.
2. **Practice explaining your notebook** – Aim for 20-25 minutes of detailed explanation.
3. **Prepare theory** – MLT, MLF, MLP concepts, especially models you used.
4. **Practice coding** – Basic pandas, numpy, sklearn, matplotlib, seaborn.
5. **Be confident** – If you don't know, say so honestly.
6. **Use documentation** – Allowed in most vivas for coding questions.
7. **Don't use LLMs** – Most proctors will deduct marks if you use LLMs during coding.
8. **Keep your notebook clean** – Well-commented, well-organized, with markdown cells.

9. **Add graphs** – Model comparison graphs, EDA graphs, confusion matrix.
10. **Know your EDA** – Insights from each graph, why you plotted it.
11. **Know your models** – How they work, parameters, pros/cons.
12. **Know your preprocessing** – Why you imputed, scaled, encoded the way you did.
13. **Know your HPT** – Parameters, ranges, time taken, improvement in score.
14. **Know your evaluation** – Why you chose the metric you did, what it means.
15. **Know your data** – Columns, types, distributions, correlations, outliers.
16. **Know your problem statement** – Classification/regression, supervised/unsupervised.
17. **Know your results** – Best model, score, leaderboard rank, why it performed best.
18. **Know your limitations** – What could be improved, what you would do differently.
19. **Know your future scope** – How you would deploy, how you would improve.
20. **Stay calm** – Take a deep breath, think before answering, and be confident.

Good luck for your viva! You've got this!